

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN
୨୨୪୪



BÁO CÁO ĐỒ ÁN MÔN HỌC

PENETRATION TESTING

KIỂM THỬ XÂM NHẬP ỨNG DỤNG

WEB WEBCLOTHESHOP

GVHD: ThS. Phạm Đình Thắng
SV thực hiện: Trần Phan Cảnh Phúc - 23DH112758
Chuyên ngành: An Ninh Mạng
Khóa: Khóa 29 - K2023

TP. Hồ Chí Minh, tháng 4 năm 2026

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển ngày càng mạnh mẽ, các ứng dụng web đã trở thành một phần không thể thiếu trong các hoạt động kinh doanh và thương mại điện tử. Tuy nhiên, cùng với sự phát triển đó, các mối đe dọa về an toàn thông tin ngày càng gia tăng, đặc biệt là các cuộc tấn công nhằm khai thác lỗ hổng bảo mật trên hệ thống web.

Vì vậy việc kiểm thử xâm nhập (Penetration Testing) đóng vai trò rất quan trọng trong việc đánh giá mức độ an toàn của hệ thống, giúp phát hiện các điểm yếu và nguy cơ tiềm ẩn trước khi bị tấn công. Thông qua quá trình pentest, các tổ chức có thể chủ động triển khai các biện pháp bảo vệ phù hợp nhằm đảm bảo tính bảo mật, toàn vẹn dữ liệu.

Trong báo cáo này, nhóm sẽ tiến hành thực hiện kiểm thử xâm nhập đối với ứng dụng web WebClothingShop, một hệ thống thương mại điện tử mô phỏng. Mục tiêu của quá trình pentest là xác định các lỗ hổng bảo mật có thể tồn tại trong hệ thống, lỗi logic nghiệp vụ và các vấn đề liên quan đến quản lý phiên làm việc.

Quá trình kiểm thử được thực hiện an toàn trong môi trường lab nhằm đánh giá tổng quan bảo mật và đề xuất giải pháp phòng thủ, qua đó giúp nhóm củng cố kiến thức và nâng cao kỹ năng thực hành xử lý lỗ hổng thực tế.

LỜI CẢM ƠN

Để kết thúc môn học Penetration Testing và có thêm nhiều hiểu biết về lĩnh vực này, em đã cơ hội thực hiện đồ án nhằm củng cố kiến thức, rèn luyện kỹ năng nghiên cứu và phát triển năng lực ứng dụng vào thực tế.

Em đã thực hiện đề tài “Pentest web WebClothingShop” dưới sự hướng dẫn tận tình và các kiến thức đã giảng dạy của thầy Phạm Đình Thắng.

Bên cạnh đó, em cũng chủ động tìm hiểu thêm các tài liệu chuyên ngành, các công cụ hỗ trợ pentest và các phương pháp tấn công phổ biến nhằm nâng cao hiệu quả đánh giá bảo mật cho ứng dụng.

Em xin gửi lời cảm ơn chân thành đến thầy Phạm Đình Thắng - người đã tận tình hướng dẫn, hỗ trợ và truyền đạt những kiến thức quý báu trong suốt quá trình học. Sự chỉ bảo của thầy là nền tảng giúp em hoàn thành tốt đề tài này.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI MỞ ĐẦU	I
LỜI CẢM ƠN	II
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN.....	III
MỤC LỤC	IV
DANH MỤC HÌNH ẢNH.....	V
DANH MỤC BẢNG.....	VI
DANH MỤC TỪ VIẾT TẮT	VII
CHƯƠNG I. CƠ SỞ LÝ THUYẾT	1
1. Tổng quan về kiểm thử bảo mật (Security Testing)	1
1.1. Khái niệm về bảo mật (Security)	1
1.2. Khái niệm và mục tiêu kiểm thử bảo mật	2
2. Tổng quan về kiểm thử xâm nhập (Penetration Testing)	3
2.1. Khái niệm Penetration Testing	3
2.2. Các phương pháp tiếp cận trong Pentest	4
CHƯƠNG II. CÁC LỖ HỔNG BẢO MẬT ỨNG DỤNG WEB	6
1. Tổ chức OWASP và OWASP Top 10:2021	6
1.1. Tổng quan về OWASP	6
1.2. OWASP Top 10:2021 và tầm quan trọng	7
2. Các lỗ hổng bảo mật liên quan đến đề tài	10
2.1. Broken Access Control (A01:2021)	10
2.2. Cross-Site Request Forgery - CSRF (A01:2021)	10
2.3. Cross-Site Scripting - XSS (A03:2021)	11
2.4. Cryptographic Failures (A02:2021)	12
2.5. Identification and Authentication Failures (A07:2021)	12

CHƯƠNG III: MÔI TRƯỜNG KIỂM THỬ VÀ CÁC CÔNG CỤ HỖ TRỢ	14
1. Giới thiệu hệ thống WebClothingShop	14
1.1. Mô tả hệ thống	14
1.2. Các chức năng chính của Web	15
2. Môi trường và công cụ kiểm thử.....	18
2.1. Môi trường thực nghiệm	18
2.2. Công cụ kiểm thử sử dụng	19
CHƯƠNG IV. THỰC NGHIỆM KIỂM THỬ WEBCLOTHINGSHOP	21
1. Thu thập thông tin hệ thống.....	21
1.1. Xác định công nghệ hệ thống (WhatWeb)	21
1.2. Dò tìm tài nguyên ẩn (dirsearch).....	21
2. Kiểm thử và khai thác lỗ hổng	22
2.1. User Enumeration – Liệt kê người dùng (A07:2021)	22
2.2. Cross-Site Request Forgery - CSRF (A01:2021).....	26
2.3. Broken Access Control - Kiểm soát truy cập bị phá vỡ (A01:2021)	29
2.4. Cross-Site Scripting - XSS (A03:2021)	31
2.5. Lưu trữ mật khẩu dạng Plaintext - Cryptographic Failures (A02:2021).....	34
3. Tổng hợp kết quả kiểm thử	34
CHƯƠNG V. ĐỀ XUẤT VÀ KHẮC PHỤC LỖ HỔNG BẢO MẬT	36
1. Khắc phục Broken Access Control (A01:2021)	36
1.1. Nguyên nhân	36
1.2. Giải pháp đề xuất	36
2. Khắc phục Cryptographic Failures - Plaintext Password (A02:2021).....	37
2.1. Nguyên nhân	37
2.2. Giải pháp đề xuất	37

3. Khắc phục CSRF - Cross-Site Request Forgery (A01:2021).....	38
3.1. Nguyên nhân	38
3.2. Giải pháp đề xuất	38
4. Khắc phục XSS - Cross-Site Scripting (A03:2021)	39
4.1. Nguyên nhân	39
4.2. Giải pháp đề xuất	39
5. Khắc phục Identification and Authentication Failures – User Enumeration (A07:2021)	41
5.1. Nguyên nhân	41
5.2. Giải pháp đề xuất	41
KẾT LUẬN	44
TÀI LIỆU THAM KHẢO	46

DANH MỤC HÌNH ẢNH

Hình 1. Mô hình CIA Triad	2
Hình 2. OWASP	6
Hình 3 Giao diện khi truy cập website.....	14
Hình 4 Giao diện đăng ký tài khoản.....	15
Hình 5 Giao diện đăng nhập.....	16
Hình 6 Chức năng Search (tìm kiếm)	16
Hình 7 Xem chi tiết sản phẩm.....	17
Hình 8 Giao diện Product của Admin	17
Hình 9 Giao diện Quản Lý đơn hàng	17
Hình 10 Giao diện quản lý người dùng.....	18
Hình 11 Giao diện xem doanh thu.....	18
Hình 12 Quét thông tin web bằng WhatWeb	21
Hình 13 Kết quả dirsearch tìm được endpoint ẩn	22
Hình 14 Kiểm thử trên giao diện login (Sai tên đăng nhập)	23
Hình 15 Kiểm thử trên giao diện login (sai mật khẩu)	24
Hình 16 Bắt Request bằng Burp Suite	24
Hình 17 Nhập Payload	25
Hình 18 Brute force các tài khoản.....	25
Hình 19 Response trả về của Server	26
Hình 20 Brute force password.....	26
Hình 21 ZAP cảnh báo CSRF	27
Hình 22 Bắt request kiểm tra CSRF trên burp suite	27
Hình 23 Tài khoản User đang đăng nhập website.....	28
Hình 24 File HTML của Hacker ép người dùng login.....	28
Hình 25 Hacker ép User vào tài khoản của Hacker	29
Hình 26 Truy cập endpoint /AdminUserServlet.....	30
Hình 27 Truy cập endpoint /AdminProductServlet.....	30
Hình 28 Nhập Script XSS vào thanh Search.....	31
Hình 29 Kết quả bị XSS.....	32

Hình 30 Nhập Script vào tên sản phẩm.....	32
Hình 31 Script chèn vào database và ghi vào html	33
Hình 32 Giao diện khi bị Stored XSS khi Clients truy cập.....	33
Hình 33 Scripts được lưu vào Database	34
Hình 34 Password lưu plaintext	34
Hình 35 Check role Admin ở doGet.....	36
Hình 36 Check role Admin doPost.....	37
Hình 37 So sánh password đã băm để login	38
Hình 38 Băm password trước khi lưu vào DB của Register	38
Hình 39 Sinh CSRF Token và Lưu Session	39
Hình 40 Kiểm tra CSRF Token trước khi login	39
Hình 41 hàm escapeHtml()	40
Hình 42 escape keyword	40
Hình 43 Khắc phục XSS	41
Hình 44 Đặt thông báo chung	42

DANH MỤC BẢNG

Bảng 1 Ba phương pháp Pentest	4
Bảng 2 Danh sách Top 10:2021	7
Bảng 3 Môi trường thực nghiệm	19
Bảng 4 Tổng hợp kết quả	34
Bảng 5 Các lỗi đề xuất khắc phục	42

DANH MỤC TỪ VIẾT TẮT

Từ Viết Tắt	Tên đầy đủ Tiếng Anh	Nghĩa Tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
BAC	Broken Access Control	Kiểm soát truy cập bị phá vỡ
CIA	Confidentiality, Integrity, Availability	Bí mật, Toàn vẹn, Sẵn sàng
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CVSS	Common Vulnerability Scoring System	Hệ thống tính điểm lỗ hổng bảo mật
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
IAF	Identification and Authentication Failures	Lỗi nhận dạng và xác thực
JSP	JavaServer Pages	Trang máy chủ Java
OSINT	Open Source Intelligence	Thu thập thông tin từ nguồn mở
OWASP	Open Web Application Security Project	Dự án bảo mật ứng dụng web mở
PTES	Penetration Testing Execution Standard	Tiêu chuẩn thực thi kiểm thử xâm nhập
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc

CHƯƠNG I. CƠ SỞ LÝ THUYẾT

1. Tổng quan về kiểm thử bảo mật (Security Testing)

1.1. Khái niệm về bảo mật (Security)

Bảo mật thông tin (Information Security) là tập hợp các biện pháp, quy trình và công nghệ được thiết kế nhằm bảo vệ thông tin và hệ thống thông tin khỏi sự truy cập, sử dụng, tiết lộ, gián đoạn, sửa đổi hoặc phá hủy trái phép. Trong bối cảnh ứng dụng web, bảo mật bao gồm việc đảm bảo các tài nguyên hệ thống - dữ liệu người dùng, mã nguồn, hạ tầng máy chủ - được bảo vệ khỏi các mối đe dọa từ cả bên trong lẫn bên ngoài tổ chức.

Mô hình CIA Triad là nền tảng lý thuyết cốt lõi của bảo mật thông tin, bao gồm ba thuộc tính chính:

- **Tính bí mật (Confidentiality):** Đảm bảo thông tin chỉ được truy cập bởi những cá nhân hoặc hệ thống được ủy quyền. Vi phạm tính bí mật xảy ra khi dữ liệu nhạy cảm bị lộ cho các bên không có quyền, ví dụ như rò rỉ thông tin tài khoản người dùng hoặc dữ liệu thẻ tín dụng.
- **Tính toàn vẹn (Integrity):** Đảm bảo thông tin không bị thay đổi, sửa đổi hoặc phá hủy trái phép trong quá trình lưu trữ và truyền tải. Dữ liệu phải chính xác, nhất quán và đáng tin cậy trong suốt vòng đời của nó.
- **Tính sẵn sàng (Availability):** Đảm bảo hệ thống và dữ liệu luôn có thể truy cập được bởi người dùng hợp lệ khi cần thiết. Các cuộc tấn công từ chối dịch vụ (DoS/DDoS) là ví dụ điển hình về vi phạm tính sẵn sàng.



Hình 1. Mô hình CIA Triad

1.2. Khái niệm và mục tiêu kiểm thử bảo mật

Kiểm thử bảo mật (**Security Testing**) là quá trình đánh giá có hệ thống một hệ thống phần mềm, ứng dụng hoặc cơ sở hạ tầng nhằm xác định các lỗ hổng, điểm yếu và rủi ro bảo mật tiềm ẩn. Mục đích không phải là đảm bảo hệ thống hoàn toàn không có lỗ hổng - một mục tiêu thực tế không thể đạt được - mà là giảm thiểu bề mặt tấn công (attack surface) và nâng cao khả năng phòng thủ tổng thể của hệ thống xuống mức rủi ro chấp nhận được.

Kiểm thử bảo mật bao gồm nhiều hình thức và phương pháp khác nhau, được lựa chọn tùy theo mục tiêu, quy mô và yêu cầu của từng dự án. Các hình thức phổ biến nhất bao gồm:

- Vulnerability Assessment (Đánh giá lỗ hổng): Quét và xác định các lỗ hổng đã biết trong hệ thống bằng các công cụ tự động, thường không bao gồm giai đoạn khai thác thực tế.
- Penetration Testing (Kiểm thử xâm nhập): Mô phỏng tấn công thực tế có kiểm soát để kiểm tra khả năng bảo vệ của hệ thống, bao gồm cả giai đoạn khai thác lỗ hổng.
- Security Auditing (Kiểm toán bảo mật): Rà soát chính sách, quy trình và cấu hình hệ thống để đảm bảo tuân thủ các tiêu chuẩn và quy định bảo mật.

- **Code Review (Đánh giá mã nguồn):** Phân tích trực tiếp mã nguồn để phát hiện các lỗi lập trình dẫn đến lỗ hổng bảo mật như SQL Injection, XSS hoặc lỗi xử lý dữ liệu đầu vào.
- **Risk Assessment (Đánh giá rủi ro):** Xác định, phân tích và ưu tiên các rủi ro bảo mật dựa trên xác suất xảy ra và mức độ tác động tiềm năng.

Mục tiêu cụ thể của kiểm thử bảo mật bao gồm: phát hiện và định danh các lỗ hổng bảo mật trước khi kẻ tấn công khai thác, đánh giá mức độ nghiêm trọng và ưu tiên xử lý, xác minh hiệu quả của các biện pháp bảo mật đã triển khai, đảm bảo tuân thủ các tiêu chuẩn bảo mật và cung cấp bằng chứng thực tế để hỗ trợ quyết định đầu tư vào bảo mật.

2. Tổng quan về kiểm thử xâm nhập (Penetration Testing)

2.1. Khái niệm Penetration Testing

Penetration Testing (kiểm thử xâm nhập), thường được gọi tắt là Pentest, là quá trình kiểm thử bảo mật có ủy quyền trong đó chuyên gia bảo mật mô phỏng các cuộc tấn công thực tế nhằm đánh giá mức độ an toàn của một hệ thống, mạng lưới hoặc ứng dụng. Khác với tấn công độc hại, pentest được thực hiện trong phạm vi hợp pháp, có sự cho phép rõ ràng từ chủ sở hữu hệ thống và tuân theo các quy tắc ứng xử nghiêm ngặt.

Penetration testing là kiểm thử bảo mật trong đó người đánh giá mô phỏng các cuộc tấn công của thế giới thực để xác định các phương pháp vượt qua các tính năng bảo mật của ứng dụng, hệ thống hoặc mạng. Đây là phương pháp kiểm thử chủ động và toàn diện nhất, cung cấp bằng chứng thực tế về mức độ rủi ro của hệ thống thay vì chỉ đưa ra các giả định lý thuyết.

Quy trình pentest tiêu chuẩn thường được chia thành **năm giai đoạn** chính theo phương pháp luận Penetration Testing Execution Standard (tiêu chuẩn thực hiện kiểm thử xâm nhập):

- **Giai đoạn 1: Thu thập thông tin (Reconnaissance):** Thu thập thông tin về mục tiêu thông qua các kỹ thuật thụ động (phân tích DNS, WHOIS) và chủ động (quét công, liệt kê dịch vụ). Đây là giai đoạn quan trọng quyết định phạm vi và hướng tấn công tiếp theo.

- **Giai đoạn 2: Quét và liệt kê (Scanning & Enumeration):** Sử dụng các công cụ tự động để xác định các cổng mở, dịch vụ đang chạy, phiên bản phần mềm và các điểm tiếp xúc tiềm năng của hệ thống mục tiêu.
- **Giai đoạn 3: Khai thác lỗ hổng (Exploitation):** Tìm kiếm và khai thác các lỗ hổng đã phát hiện để xâm nhập vào hệ thống. Giai đoạn này phải được thực hiện cẩn thận để tránh gây ảnh hưởng đến hoạt động của hệ thống thực tế.
- **Giai đoạn 4: Duy trì quyền truy cập (Post-Exploitation):** Đánh giá mức độ thiệt hại có thể xảy ra sau khi xâm nhập thành công, bao gồm khả năng leo thang đặc quyền và truy cập dữ liệu nhạy cảm.
- **Giai đoạn 5: Báo cáo (Reporting):** Tổng hợp toàn bộ phát hiện, phân tích mức độ nghiêm trọng và đề xuất các biện pháp khắc phục cụ thể, có thể thực thi được.

2.2. Các phương pháp tiếp cận trong Pentest

Trong thực tiễn kiểm thử xâm nhập, có **ba phương pháp** tiếp cận chính được phân loại dựa trên mức độ thông tin mà kiểm thử viên được cung cấp về hệ thống mục tiêu trước khi bắt đầu. Mỗi phương pháp có ưu điểm, nhược điểm riêng và phù hợp với các tình huống kiểm thử khác nhau.

Bảng 1. Ba phương pháp Pentest

Phương pháp	Mức độ thông tin	Ưu điểm	Nhược điểm
Black-box Testing	Không có thông tin nội bộ, chỉ biết địa chỉ mục tiêu.	Mô phỏng chính xác góc nhìn kẻ tấn công từ bên ngoài, kết quả thực tế.	Tốn nhiều thời gian, có thể bỏ sót lỗ hổng sâu bên trong hệ thống.
White-box Testing	Toàn quyền truy cập mã nguồn, tài liệu kiến trúc, thông tin hạ tầng.	Kiểm tra toàn diện, phát hiện lỗ hổng ở mức mã nguồn và thiết kế.	Không phản ánh thực tế tấn công từ bên ngoài, yêu cầu tài nguyên lớn.

Gray-box Testing	Có thông tin nội bộ như tài khoản người dùng, tài liệu hệ thống, hoặc quyền truy cập hạn chế vào mã nguồn / cơ sở dữ liệu.	Cân bằng giữa thực tế và độ bao phủ, hiệu quả về thời gian và chi phí.	Kết quả phụ thuộc nhiều vào chất lượng thông tin được cung cấp ban đầu.
---------------------	--	--	---

Ngoài ba phương pháp tiếp cận cơ bản trên, trong thực tiễn còn có một số hình thức kiểm thử xâm nhập chuyên biệt khác được phân loại theo đối tượng mục tiêu. Kiểm thử xâm nhập ứng dụng web (Web Application Pentest) tập trung vào các lỗ hổng đặc thù của ứng dụng web như SQL Injection, XSS, CSRF và các lỗ hổng trong OWASP Top 10. Kiểm thử xâm nhập mạng (Network Pentest) đánh giá bảo mật của hạ tầng mạng, bao gồm tường lửa, bộ định tuyến và các dịch vụ mạng. Kiểm thử xâm nhập ứng dụng di động (Mobile Pentest) tập trung vào các ứng dụng iOS và Android với các rủi ro đặc thù như lưu trữ dữ liệu không an toàn và giao tiếp mạng không được mã hóa.

Trong phạm vi đề tài này, nhóm áp dụng phương pháp Gray-box Testing đối với ứng dụng web WebClothingShop. Phương pháp này được lựa chọn vì cho phép kết hợp giữa kiểm thử từ góc nhìn người dùng hợp lệ và việc sử dụng thông tin nội bộ (như mã nguồn và cơ sở dữ liệu). Nhờ đó, quá trình kiểm thử vừa phản ánh được thực tế của kẻ tấn công, vừa nâng cao khả năng phát hiện các lỗ hổng bảo mật trong hệ thống.

CHƯƠNG II. CÁC LỖ HỔNG BẢO MẬT ỨNG DỤNG WEB

1. Tổ chức OWASP và OWASP Top 10:2021

1.1. Tổng quan về OWASP

OWASP (Open Web Application Security Project) là một tổ chức phi lợi nhuận quốc tế được thành lập vào năm 2001, hoạt động với mục tiêu cải thiện bảo mật phần mềm, đặc biệt là các ứng dụng web. OWASP cung cấp các tài nguyên miễn phí và mở, bao gồm tài liệu, công cụ, tiêu chuẩn và các hướng dẫn thực tiễn tốt nhất trong lĩnh vực bảo mật ứng dụng.

Tổ chức này quy tụ hàng nghìn chuyên gia bảo mật, nhà nghiên cứu và lập trình viên từ khắp nơi trên thế giới, cùng nhau xây dựng một cộng đồng kiến thức mở. Các dự án nổi bật của OWASP bao gồm OWASP Top 10, OWASP Testing Guide, OWASP ASVS (Application Security Verification Standard) và nhiều công cụ kiểm thử bảo mật được sử dụng rộng rãi trong ngành công nghiệp.



Hình 2. OWASP

1.2. OWASP Top 10:2021 và tầm quan trọng

OWASP Top 10 là danh sách mười lỗi hỏng bảo mật phổ biến và nguy hiểm nhất trong ứng dụng web, được OWASP công bố định kỳ dựa trên dữ liệu thu thập từ hàng trăm tổ chức và hàng triệu ứng dụng thực tế trên toàn thế giới. Và tiếp tục là tài liệu tham chiếu chuẩn mực được sử dụng rộng rãi trong cộng đồng bảo mật.

Danh sách này không chỉ đơn thuần là thống kê các lỗi hỏng phổ biến, mà còn phản ánh mức độ nghiêm trọng về khả năng bị khai thác, phạm vi ảnh hưởng và tác động đến hệ thống. Đây là tài liệu nền tảng để các nhà phát triển, kỹ sư bảo mật và kiểm thử viên xâm nhập tập trung kiểm tra và xử lý các rủi ro bảo mật ưu tiên.

Bảng 2 Danh sách Top 10:2021

STT	Mã lỗi hỏng	Tên lỗi hỏng	Mô tả ngắn gọn
1	A01	Broken Access Control (Lỗi kiểm soát truy cập)	Xảy ra khi ứng dụng không thực thi đúng chính sách phân quyền, cho phép người dùng truy cập tài nguyên hoặc thực hiện hành động ngoài phạm vi quyền hạn. Kẻ tấn công có thể thay đổi tham số URL, cookie hoặc token để leo thang đặc quyền, truy cập dữ liệu người dùng khác hoặc thực hiện chức năng quản trị trái phép.
2	A02	Cryptographic Failures (Lỗi mã hóa)	Xảy ra khi ứng dụng sử dụng thuật toán mã hóa yếu, lỗi thời hoặc hoàn toàn không mã hóa dữ liệu nhạy cảm như mật khẩu, số thẻ tín dụng, thông tin cá nhân. Hậu quả là dữ liệu bị lộ khi truyền qua mạng hoặc khi cơ sở dữ liệu bị tấn công, gây vi phạm nghiêm trọng quyền riêng tư người dùng.

3	A03	Injection	Xảy ra khi ứng dụng không kiểm tra và lọc đầu vào của người dùng, cho phép kẻ tấn công chèn mã độc hại như SQL Injection, XSS, NoSQL Injection, OS Command Injection vào câu lệnh truy vấn. Hậu quả có thể bao gồm đánh cắp toàn bộ dữ liệu, xóa cơ sở dữ liệu hoặc chiếm quyền kiểm soát máy chủ.
4	A04	Insecure Design (Thiết kế không an toàn)	Phát sinh từ giai đoạn thiết kế khi kiến trúc hệ thống không tích hợp các nguyên tắc bảo mật từ đầu (Security by Design). Không có các cơ chế kiểm soát bảo mật phù hợp trong luồng nghiệp vụ, dẫn đến các lỗ hổng logic nghiệp vụ mà không thể khắc phục chỉ bằng cách vá lỗi thông thường.
5	A05	Security Misconfiguration (Lỗi cấu hình bảo mật)	Khi các thành phần hệ thống như máy chủ web, cơ sở dữ liệu, framework hoặc dịch vụ đám mây được cấu hình sai hoặc để mặc định không an toàn. Biểu hiện thường gặp là để lộ thông báo lỗi chi tiết, bật tính năng không cần thiết, sử dụng tài khoản mặc định hoặc phân quyền quá rộng.
6	A06	Vulnerable & Outdated Components (Các thành phần dễ bị tổn thương và lỗi thời)	Xảy ra khi ứng dụng sử dụng các thư viện, framework, module hoặc hệ điều hành đã lỗi thời, chứa các lỗ hổng bảo mật đã được công bố. Kẻ tấn công có thể tra cứu và khai thác các lỗ hổng này một cách có hệ thống, đặc

			biệt nguy hiểm khi ứng dụng không có quy trình cập nhật phụ thuộc định kỳ.
7	A07	Identification & Authentication Failures (Lỗi nhận dạng và xác thực)	Xảy ra khi cơ chế xác thực và quản lý phiên bị triển khai yếu, cho phép kẻ tấn công giả mạo danh tính người dùng. Biểu hiện thường gặp gồm cho phép mật khẩu yếu, không giới hạn số lần đăng nhập sai, quản lý session token không an toàn, lộ thông tin tài khoản qua thông báo lỗi phân biệt.
8	A08	Software & Data Integrity Failures (Lỗi phần mềm và tính toàn vẹn dữ liệu)	Xảy ra khi ứng dụng hoặc hệ thống CI/CD không xác minh tính toàn vẹn của phần mềm, plugin, thư viện hoặc dữ liệu trước khi sử dụng. Kẻ tấn công có thể chèn mã độc vào quá trình cập nhật phần mềm tự động hoặc thay thế các gói phụ thuộc bằng phiên bản bị can thiệp mà hệ thống không phát hiện.
9	A09	Security Logging & Monitoring Failures (Lỗi ghi nhật ký và giám sát bảo mật)	Xảy ra khi ứng dụng không ghi nhật ký đầy đủ các sự kiện bảo mật quan trọng hoặc không có cơ chế giám sát và cảnh báo kịp thời. Điều này khiến các cuộc tấn công có thể diễn ra trong thời gian dài mà không bị phát hiện, làm tăng đáng kể thời gian phản ứng sự cố và mức độ thiệt hại.
10	A10	Server-Side Request Forgery (Tấn công giả)	Xảy ra khi ứng dụng cho phép người dùng cung cấp URL và thực hiện yêu cầu HTTP đến địa chỉ đó mà không kiểm tra hợp lệ. Kẻ tấn công có thể lợi dụng để truy cập dịch vụ nội

		mạo yêu cầu phía máy chủ)	bộ, quét mạng nội bộ, đọc metadata đám mây hoặc vượt qua tường lửa để tấn công các hệ thống không được phép truy cập từ bên ngoài.
--	--	---------------------------	--

2. Các lỗ hổng bảo mật liên quan đến đề tài

Trong quá trình kiểm thử xâm nhập hệ thống WebClothingShop, nhóm thực hiện đề tài đã phát hiện năm loại lỗ hổng bảo mật thuộc các nhóm trong OWASP Top 10:2021. Mục này trình bày chi tiết cơ chế, nguyên nhân và hậu quả của từng loại lỗ hổng nhằm làm cơ sở lý thuyết cho phân thực nghiệm.

2.1. Broken Access Control (A01:2021)

Broken Access Control (**Kiểm soát truy cập bị phá vỡ**) được xếp hạng số 1 trong OWASP Top 10:2021, phản ánh mức độ phổ biến và nguy hiểm hàng đầu của lỗ hổng này. Đây là lỗ hổng xảy ra khi ứng dụng không thực thi đúng các chính sách phân quyền, cho phép người dùng thực hiện các hành động hoặc truy cập tài nguyên nằm ngoài phạm vi quyền hạn được cấp.

Nguyên nhân chính của lỗ hổng này bao gồm: kiểm tra quyền truy cập chỉ ở phía client (client-side), không kiểm tra quyền ở phía server với mỗi yêu cầu, cho phép leo thang đặc quyền thông qua thay đổi tham số URL hoặc cookie, hoặc cấu hình sai các phân vùng bảo vệ trong ứng dụng.

Hậu quả có thể bao gồm: truy cập trái phép vào dữ liệu của người dùng khác, thực hiện chức năng quản trị mà không có quyền, đọc hoặc chỉnh sửa dữ liệu nhạy cảm và trong nhiều trường hợp có thể dẫn đến chiếm quyền điều khiển toàn bộ hệ thống.

2.2. Cross-Site Request Forgery - CSRF (A01:2021)

Cross-Site Request Forgery gọi là tấn công giả mạo yêu cầu chéo trang, là một dạng tấn công thuộc nhóm Broken Access Control trong OWASP Top 10:2021. Cuộc tấn công này xảy ra khi kẻ tấn công lừa trình duyệt của nạn nhân gửi các yêu cầu HTTP độc hại đến một ứng dụng web mà nạn nhân đang đăng nhập, mà không cần biết thông tin xác thực của nạn nhân.

Cơ chế hoạt động của CSRF dựa trên việc trình duyệt tự động đính kèm cookie phiên (session cookie) vào mọi yêu cầu gửi đến tên miền đích. Kẻ tấn công chỉ cần tạo ra một trang web hoặc email chứa mã HTML/JavaScript tự động gửi yêu cầu HTTP (GET hoặc POST) đến ứng dụng mục tiêu. Khi nạn nhân truy cập trang độc hại trong khi đang đăng nhập, yêu cầu sẽ được thực thi với quyền hạn của nạn nhân.

Hậu quả của CSRF có thể rất nghiêm trọng như: thay đổi mật khẩu, email tài khoản, thực hiện giao dịch tài chính trái phép, thay đổi thông tin cấu hình hệ thống, hoặc thực hiện bất kỳ hành động nào mà nạn nhân được phép làm trên ứng dụng. Biện pháp phòng chống phổ biến nhất là sử dụng CSRF Token - một giá trị ngẫu nhiên, bí mật được nhúng vào mỗi biểu mẫu và xác minh tại phía máy chủ.

2.3. Cross-Site Scripting - XSS (A03:2021)

Cross-Site Scripting (Chèn mã độc vào trang web) là một trong những lỗ hổng bảo mật phổ biến nhất trên ứng dụng web, được phân loại thuộc nhóm Injection trong OWASP Top 10:2021. XSS xảy ra khi ứng dụng web nhận dữ liệu đầu vào không tin cậy từ người dùng và nhúng trực tiếp vào trang HTML mà không qua quá trình kiểm tra hoặc mã hóa đúng cách, cho phép kẻ tấn công chèn mã JavaScript độc hại vào trang web hiển thị cho người dùng khác.

Có ba dạng XSS chính:

(1) Reflected XSS - mã độc được phản chiếu ngay lập tức trong phản hồi HTTP từ server, thường được khai thác thông qua liên kết độc hại.

(2) Stored XSS - mã độc được lưu trữ trong cơ sở dữ liệu và thực thi mỗi khi người dùng xem trang chứa nội dung đó

(3) DOM-based XSS - mã độc được thực thi trực tiếp trong môi trường DOM (Document Object Model) phía client mà không cần tương tác với server.

Hậu quả của XSS bao gồm: đánh cắp cookie phiên và chiếm đoạt tài khoản (session hijacking), chuyển hướng người dùng đến trang web độc hại, thu thập thông tin nhạy cảm (keylogging), thực hiện các hành động thay mặt người dùng hoặc thậm chí phát tán mã độc (malware). Biện pháp phòng chống bao gồm: mã hóa đầu vào (input encoding),

sử dụng Content Security Policy (CSP) và áp dụng các thư viện lọc đầu vào được kiểm chứng.

2.4. Cryptographic Failures (A02:2021)

Cryptographic Failures (**Lỗi mã hóa**) trong OWASP Top 10:2021 liên quan đến tất cả các thất bại trong việc bảo vệ dữ liệu nhạy cảm thông qua mã hóa. Đây là một trong những sai lầm bảo mật nghiêm trọng nhất trong lập trình ứng dụng, khi mật khẩu người dùng được lưu trực tiếp vào cơ sở dữ liệu mà không qua bất kỳ quá trình băm (hash) hay mã hóa nào.

Theo các tiêu chuẩn bảo mật hiện đại, mật khẩu phải được lưu trữ dưới dạng giá trị băm một chiều sử dụng các thuật toán được thiết kế đặc biệt cho mục đích này như bcrypt hoặc Argon2. Các thuật toán này có đặc điểm chậm cố ý và hỗ trợ cơ chế **Salt** (thêm một chuỗi ký tự ngẫu nhiên, duy nhất vào mật khẩu trước khi băm) để chống lại các tấn công brute-force và rainbow table.

Khi ứng dụng lưu mật khẩu dạng plaintext, nếu kẻ tấn công có thể truy cập cơ sở dữ liệu thông qua SQL Injection, toàn bộ thông tin đăng nhập của tất cả người dùng sẽ bị lộ ngay lập tức mà không cần bất kỳ nỗ lực bẻ khóa nào. Điều này vi phạm nghiêm trọng quyền riêng tư người dùng, đặc biệt vì nhiều người sử dụng cùng một mật khẩu cho nhiều dịch vụ khác nhau.

2.5. Identification and Authentication Failures (A07:2021)

Identification and Authentication Failures (**Nhận dạng và xác thực bị hỏng**) trong OWASP Top 10:2021. Lỗi hỏng này cho phép kẻ tấn công xác định xem một tên người dùng hoặc địa chỉ email có tồn tại trong hệ thống hay không, thông qua việc phân tích sự khác biệt trong phản hồi của ứng dụng khi đăng nhập, đăng ký hoặc khôi phục mật khẩu.

Cơ chế khai thác thường dựa vào việc ứng dụng trả về các thông báo lỗi khác nhau, chẳng hạn như "Tên đăng nhập không tồn tại" và "Mật khẩu không chính xác" thay vì một thông báo chung chung. Ngoài ra, sự khác biệt về thời gian phản hồi (timing attack) giữa trường hợp tài khoản tồn tại và không tồn tại cũng có thể bị lợi dụng.

Khi kẻ tấn công có được danh sách tài khoản hợp lệ, họ có thể sử dụng thông tin này để thực hiện các cuộc tấn công tiếp theo như brute-force, credential stuffing (sử dụng thông tin đăng nhập bị rò rỉ từ các vụ vi phạm dữ liệu khác) hoặc các cuộc tấn công phi kỹ

thuật (social engineering) nhắm vào đúng đối tượng người dùng. Biện pháp phòng chống là sử dụng thông báo lỗi chung cho tất cả các trường hợp thất bại xác thực và áp dụng cơ chế giới hạn tốc độ (rate limiting).

CHƯƠNG III: MÔI TRƯỜNG KIỂM THỬ VÀ CÁC CÔNG CỤ HỖ TRỢ

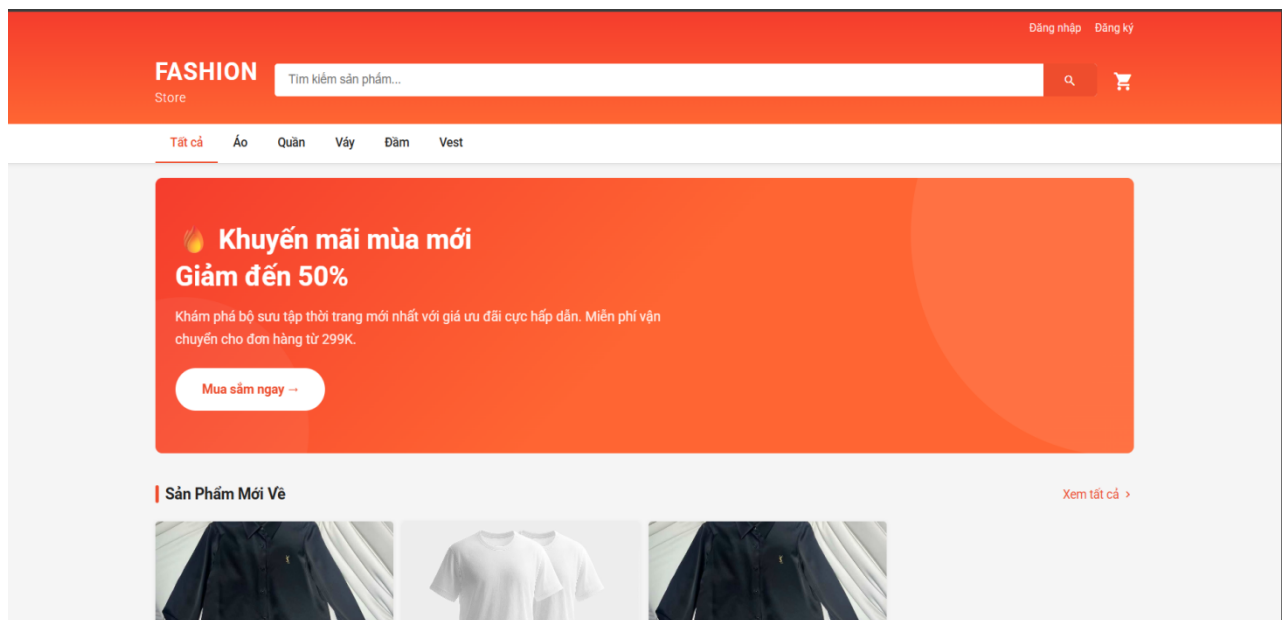
1. Giới thiệu hệ thống WebClothingShop

1.1. Mô tả hệ thống

WebClothingShop là một ứng dụng web thương mại điện tử được xây dựng nhằm phục vụ nhu cầu mua sắm thời trang trực tuyến. Hệ thống cho phép người dùng thực hiện các chức năng cơ bản như đăng ký tài khoản, đăng nhập, tìm kiếm và xem thông tin sản phẩm, cũng như quản lý giỏ hàng và thực hiện các thao tác mua sắm.

Hệ thống còn cung cấp khu vực quản trị dành cho quản trị viên (admin), cho phép thực hiện các chức năng như quản lý sản phẩm, đơn hàng và người dùng. Điều này tạo nên một môi trường đầy đủ cho việc mô phỏng và kiểm thử các kịch bản bảo mật trong ứng dụng web.

Ứng dụng được xây dựng trên nền tảng Java với mô hình Servlet/JSP và được triển khai trên máy chủ ứng dụng Apache Tomcat.

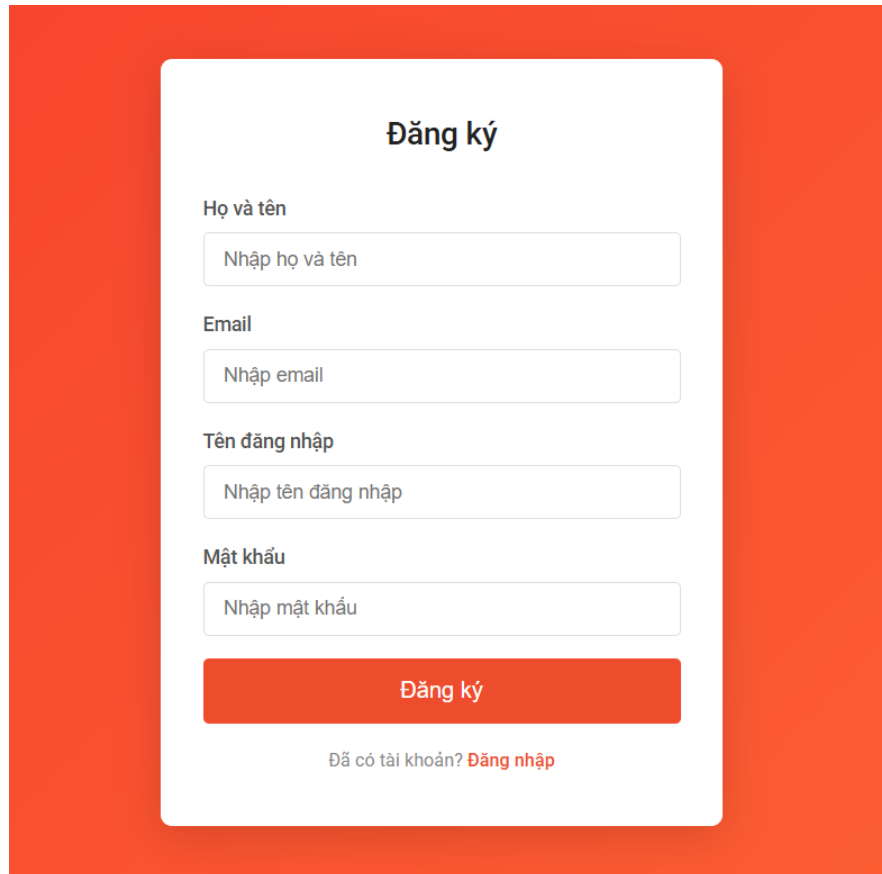


Hình 3 Giao diện khi truy cập website

1.2. Các chức năng chính của Web

1. Người dùng (User)

- Đăng ký



Đăng ký

Họ và tên
Nhập họ và tên

Email
Nhập email

Tên đăng nhập
Nhập tên đăng nhập

Mật khẩu
Nhập mật khẩu

Đăng ký

Đã có tài khoản? [Đăng nhập](#)

Hình 4 Giao diện đăng ký tài khoản

- Đăng nhập

Đăng nhập

Tên đăng nhập

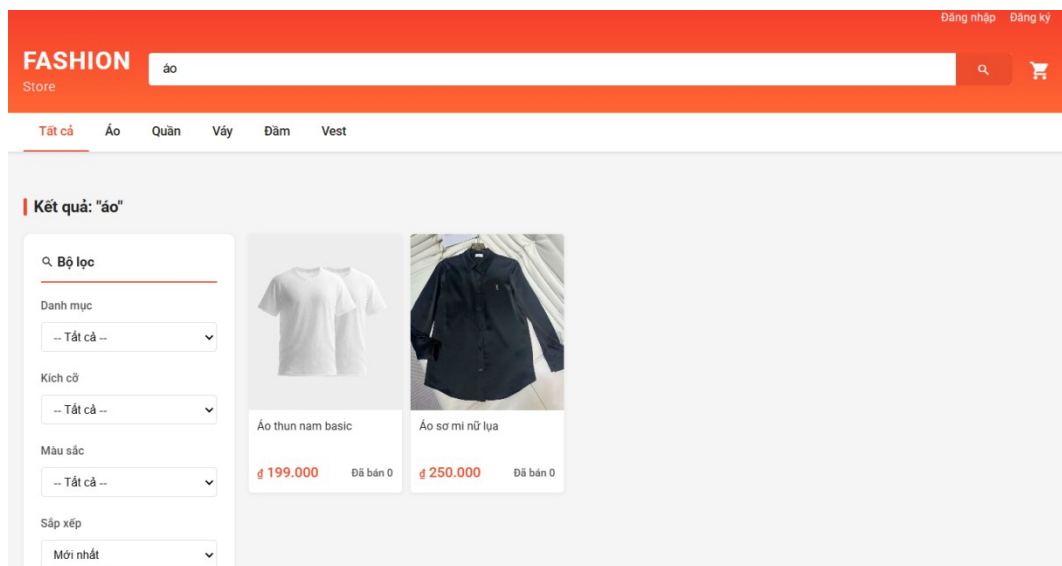
Mật khẩu

Đăng nhập

Bạn mới biết đến Fashion Store? [Đăng ký](#)

Hình 5 Giao diện đăng nhập

- Tìm kiếm sản phẩm



Hình 6 Chức năng Search (tìm kiếm)

- Xem chi tiết sản phẩm



Hình 7 Xem chi tiết sản phẩm

2. Quản trị viên (Admin)

- Quản lý sản phẩm

Quản Lý Sản Phẩm					
					+ Thêm Sản Phẩm ← Dashboard
ID	ẢNH	TÊN SẢN PHẨM	GIÁ	DANH MỤC	THAO TÁC
1		Áo thun nam basic	199.000đ	Áo	Xem Sửa Xóa
2		Áo sơ mi nữ lụa	250.000đ	Áo	Xem Sửa Xóa
3		vmware	30.000đ	Áo	Xem Sửa Xóa

Hình 8 Giao diện Product của Admin

- Quản lý đơn hàng

Quản Lý Đơn Hàng						
						← Dashboard
MÃ ĐƠN	KHÁCH HÀNG	NGÀY ĐẶT	TỔNG TIỀN	THANH TOÁN	TRẠNG THÁI	THAO TÁC

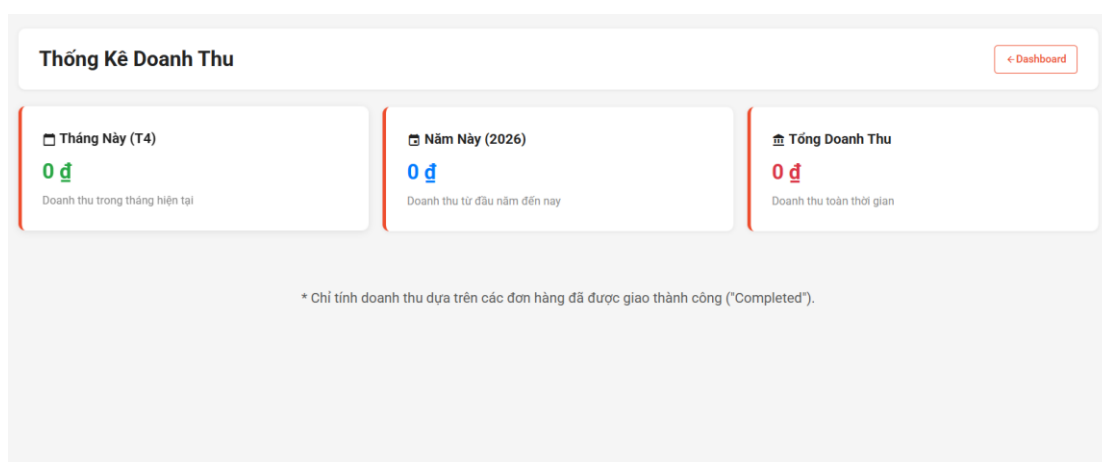
Hình 9 Giao diện Quản Lý đơn hàng

- Quản lý người dùng

ID	HỌ TÊN	EMAIL	DIỆN THOẠI	VAI TRÒ	TRẠNG THÁI	THAO TÁC
1	Administrator	admin@gmail.com	0123456789	Admin	Hoạt động	Admin
2	Nguyễn Văn A	customer1@gmail.com	0987654321	User	Hoạt động	Xem Khóa
220		hack@gmail.com		User	Hoạt động	Xem Khóa
221	ZAP	zaproxy@example.com		User	Hoạt động	Xem Khóa
222	ZAP	zaproxy@example.com		User	Hoạt động	Xem Khóa

Hình 10 Giao diện quản lý người dùng

- Doanh thu



Hình 11 Giao diện xem doanh thu

2. Môi trường và công cụ kiểm thử

2.1. Môi trường thực nghiệm

Ứng dụng WebClothingShop được triển khai cục bộ thông qua NetBeans 21 với Apache Tomcat tích hợp sẵn, kết nối đến SQL Server 2022. Môi trường cục bộ giúp kiểm thử an toàn, không ảnh hưởng đến hệ thống thật, đồng thời cho phép quan sát hành vi ứng dụng ở cấp độ mã nguồn theo phương pháp Gray-box.

Bảng 3 Môi trường thực nghiệm

Thành phần	Thông số	Ghi chú
Trình duyệt	Google Chrome	Trình duyệt kiểm thử chính
Ứng dụng mục tiêu	WebClothingShop	Website bán quần áo trực tuyến
Ngôn ngữ lập trình	Java Servlet/JSP	Không sử dụng framework Spring
Cơ sở dữ liệu	SQL Server 2022	Lưu trữ dữ liệu người dùng, sản phẩm
Môi trường chạy	Apache Tomcat	Web container cho Java Servlet/JSP
IDE	NetBeans 21	Build và deploy ứng dụng trên localhost
Địa chỉ truy cập	http://localhost:8080/WebClothingShop	Chạy cục bộ, không cần internet

2.2. Công cụ kiểm thử sử dụng

Sử dụng công cụ chuyên biệt, mỗi công cụ phục vụ một giai đoạn hoặc mục tiêu kiểm thử cụ thể. Tất cả đều là phần mềm mã nguồn mở hoặc bản miễn phí, được sử dụng rộng rãi trong cộng đồng bảo mật chuyên nghiệp.

Vai trò của từng công cụ trong quy trình kiểm thử:

- **WhatWeb:** thực hiện ở giai đoạn Reconnaissance để xác định công nghệ, server, framework của WebClothingShop trước khi tiến hành kiểm thử chuyên sâu.

- **dirsearch:** quét các đường dẫn ẩn, trang admin, file cấu hình nhằm phát hiện endpoint không được bảo vệ.
- **Burp Suite:** công cụ trung tâm, chặn và phân tích toàn bộ HTTP traffic để kiểm tra Broken Access Control, CSRF và User Enumeration.
- **OWASP ZAP:** quét tự động phát hiện các cấu hình bảo mật sai trên toàn bộ ứng dụng.
- **sqlmap:** xác minh SQL Injection và kiểm tra cách ứng dụng lưu trữ mật khẩu trong cơ sở dữ liệu.

CHƯƠNG IV. THỰC NGHIỆM KIỂM THỬ WEBCLOTHINGSHOP

1. Thu thập thông tin hệ thống

Trước khi tiến hành khai thác các lỗ hổng cụ thể, giai đoạn thu thập thông tin (Reconnaissance) được thực hiện nhằm xác định bề mặt tấn công của hệ thống WebClothingShop. Giai đoạn này sử dụng hai công cụ là WhatWeb và dirsearch để thu thập thông tin về công nghệ sử dụng và các tài nguyên ẩn của ứng dụng.

1.1. Xác định công nghệ hệ thống (WhatWeb)

Công cụ được sử dụng trong bước này là **WhatWeb** để tiến hành quét các thành phần công nghệ của website. Tiến hành quét thông tin hệ thống bằng câu lệnh trong cmd:

```
ruby whatweb -a 3 http://127.0.0.1:8080/WebClothingShop
```

Kết quả: Ứng dụng sử dụng ngôn ngữ Java, cơ chế quản lý phiên JSESSION ID. Có thiết lập HTTPOnly để hạn chế truy cập từ client.

```
Phuc@PhucDepTrai MINGW64 /d/whatweb (master)
$ ruby whatweb -a 3 http://127.0.0.1:8080/WebClothingShop
http://127.0.0.1:8080/WebClothingShop [302 Found] Country[RESERVED][ZZ], IP[127.0.0.1], RedirectLocation[/WebClothingShop/]
http://127.0.0.1:8080/WebClothingShop/ [302 Found] Cookies[JSESSIONID], Country[RESERVED][ZZ], HttpOnly[JSESSIONID], IP[127.0.0.1], Java, RedirectLocation[/WebClothingShop/home]
http://127.0.0.1:8080/WebClothingShop/home [200 OK] Country[RESERVED][ZZ], HTML5, IP[127.0.0.1], Title[Fashion Store - Mua sắm thời trang online]
Phuc@PhucDepTrai MINGW64 /d/whatweb (master)
$
```

Hình 12 Quét thông tin web bằng WhatWeb

1.2. Dò tìm tài nguyên ẩn (dirsearch)

dirsearch được sử dụng để quét các đường dẫn URL ẩn, trang quản trị và file cấu hình không được liên kết công khai trên ứng dụng. Mục tiêu là phát hiện các endpoint có thể

bị truy cập trái phép, từ đó định hướng cho việc kiểm thử Broken Access Control. Lệnh thực thi:

```
python3 dirsearch.py -u http://127.0.0.1:8080/WebClothingShop -e jsp,java,html,txt
```

Kết quả: phát hiện một số đường dẫn quan trọng bao gồm các trang quản trị dành cho admin.



```
[14:05:23] 200 - 30KB - /WebClothingShop/home
Added to the queue: webClothingShop/home/
[14:05:24] 200 - 1KB - /WebClothingShop/login.jsp
Added to the queue: webClothingShop/login.jsp/
[14:05:25] 200 - 7KB - /WebClothingShop/profile.jsp
Added to the queue: webClothingShop/profile.jsp/
[14:05:25] 200 - 2KB - /WebClothingShop/register.jsp
Added to the queue: webClothingShop/register.jsp/
[14:05:26] 302 - 0B - /WebClothingShop/AdminOrderServlet -> /webClothingShop/login.jsp
[14:05:26] 302 - 0B - /WebClothingShop/AdminRevenueServlet -> /webClothingShop/login.jsp
[14:05:26] 200 - 3KB - /WebClothingShop/CartServlet
[14:05:26] 200 - 8KB - /WebClothingShop/AdminProductServlet
[14:05:26] 200 - 13KB - /WebClothingShop/AdminUserServlet
Added to the queue: webClothingShop/CartServlet/
Added to the queue: webClothingShop/AdminProductServlet/
Added to the queue: webClothingShop/AdminUserServlet/
[14:05:27] Scanning: webClothingShop/admin/
[14:05:28] 302 - 0B - /WebClothingShop/admin/ -> /webClothingShop/home
```

Hình 13 Kết quả dirsearch tìm được endpoint ẩn

2. Kiểm thử và khai thác lỗ hổng

Dựa trên tiêu chuẩn PTES, quá trình kiểm thử được triển khai theo hướng tiếp cận có hệ thống nhằm đảm bảo tính đầy đủ và chính xác. Với mỗi lỗ hổng được phát hiện, các bước thực hiện bao gồm:

- Xác định điểm tấn công tiềm năng thông qua việc phân tích chức năng và bề mặt tấn công của ứng dụng.
- Thực hiện kiểm thử và khai thác bằng các công cụ và kỹ thuật phù hợp để xác minh khả năng khai thác thực tế
- Ghi nhận kết quả, thu thập bằng chứng và đánh giá mức độ ảnh hưởng của lỗ hổng trong bối cảnh tổng thể của hệ thống.

2.1. User Enumeration – Liệt kê người dùng (A07:2021)

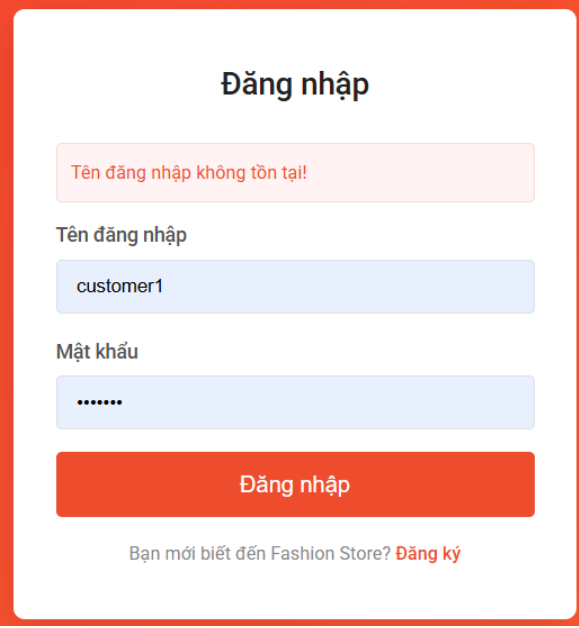
1. Xác định điểm tấn công:

Chức năng đăng nhập tại trang login là điểm tấn công chính. Khi người dùng nhập thông tin đăng nhập sai, ứng dụng trả về các thông báo lỗi khác nhau tùy thuộc vào việc tên người dùng có tồn tại trong hệ thống hay không, thay vì sử dụng một thông báo chung chung cho mọi trường hợp thất bại.

2. Thực hiện kiểm thử trên giao diện:

Trên giao diện thử nghiệm với 2 trường hợp: Nhập tên đăng nhập không tồn tại hoặc Nhập tên đăng nhập có tồn tại nhưng sai mật khẩu.

- **Trường hợp 1:** Tên đăng nhập không tồn tại, web trả về thông báo “Tên đăng nhập không tồn tại!”.



The screenshot shows a login interface with the title "Đăng nhập". Below the title is a red error message box stating "Tên đăng nhập không tồn tại!". The form contains two input fields: "Tên đăng nhập" (Username) with the text "customer1" and "Mật khẩu" (Password) with masked characters ".....". A red button labeled "Đăng nhập" is positioned below the password field. At the bottom of the form, there is a link that says "Bạn mới biết đến Fashion Store? [Đăng ký](#)".

Hình 14 Kiểm thử trên giao diện login (Sai tên đăng nhập)

- **Trường hợp 2:** Nhập tên đăng nhập tồn tại nhưng sai passwd, web trả về thông báo “Mật khẩu không chính xác!”.

Đăng nhập

Mật khẩu không chính xác!

Tên đăng nhập

customer1

Mật khẩu

.....

Đăng nhập

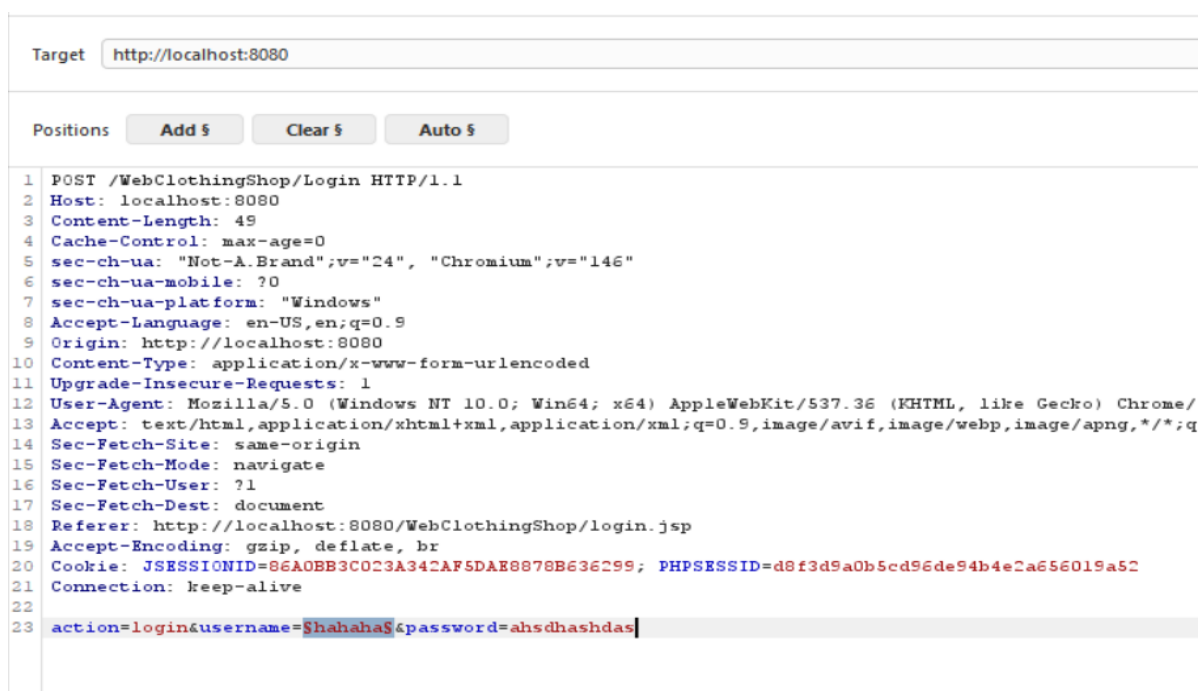
Bạn mới biết đến Fashion Store? [Đăng ký](#)

Hình 15 Kiểm thử trên giao diện login (sai mật khẩu)

3. Kết quả: Sự khác biệt rõ ràng trong thông báo lỗi giữa hai trường hợp xác nhận lỗi hỏng User Enumeration tồn tại. Kẻ tấn công có thể lợi dụng điều này để xây dựng danh sách tài khoản hợp lệ, sau đó sử dụng danh sách này cho các cuộc tấn công brute-force.

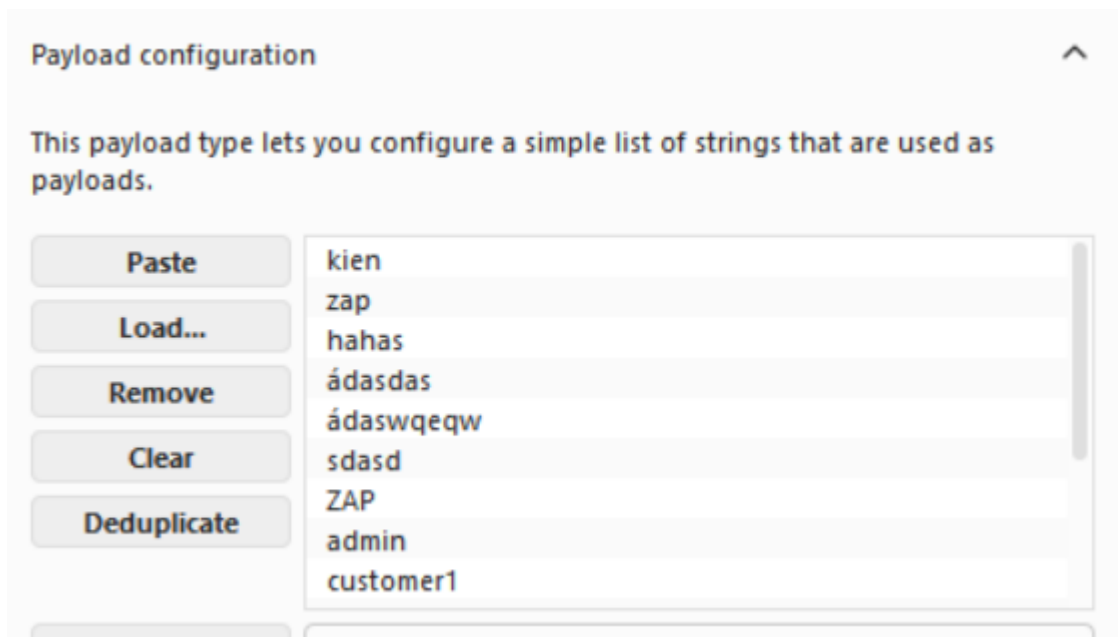
4. Thực hiện brute-force username bằng Burp suite:

- Bắt Request login gửi đến Intruder và thêm dấu "\$" vào tên user để brute-force.



Hình 16 Bắt Request bằng Burp Suite

- Nhập các tên user tạo sẵn vào Payload, sau đó chọn start attack để bắt đầu.



Hình 17 Nhập Payload

- **Kết quả:** Dựa vào status code, có độ dài length thấp và đọc response thì sẽ xác nhận được những tài khoản nào tồn tại.

Payload	Status code	Response received	Error	Timeout	Length
	200	35			1507
kien	200	8			1507
zap	200	8			1501
hahas	200	7			1507
adasdas	400	61			3520
adaswqeqw	400	3			3524
sdasd	200	3			1507
ZAP	200	5			1501
admin	200	9			1501
customer1	200	6			1501
thong	200	10			1507
cuong	200	9			1507
phuc	200	3			1507
duc	200	3			1507
lam	200	3			1507
chuong	200	9			1507

Hình 18 Brute force các tài khoản

```

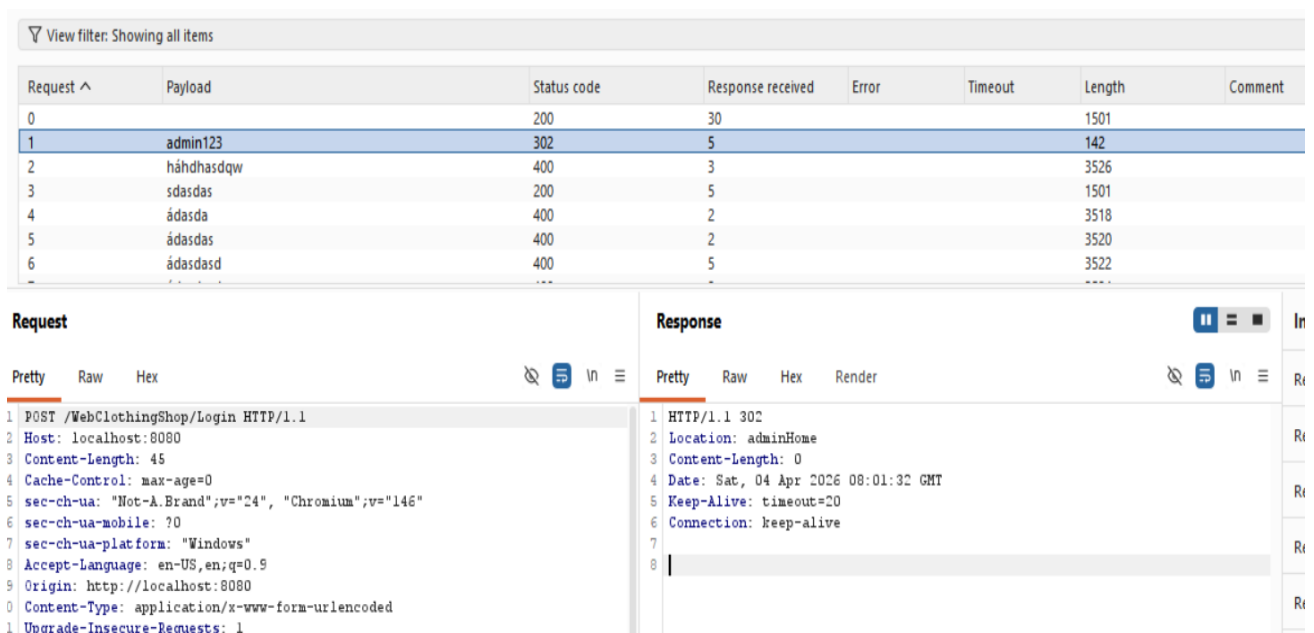
16 <link rel="stylesheet" href="/WebClothingShop/css/style.css">
17 </head>
18 <body>
19 <div class="auth-wrapper">
20 <div class="auth-card">
21 <h2>
    Đăng nhập
  </h2>
22
23
24 <div class="error-msg">
    Mật khẩu không chính xác!
  </div>
25
26
27 <form action="/WebClothingShop/Login" method="post">
28 <input type="hidden" name="action" value="login"/>

```

Hình 19 Response trả về của Server

5. Thực hiện brute-force password bằng Burp suite:

- Dựa vào username đã tìm được, tiến hành brute-force password



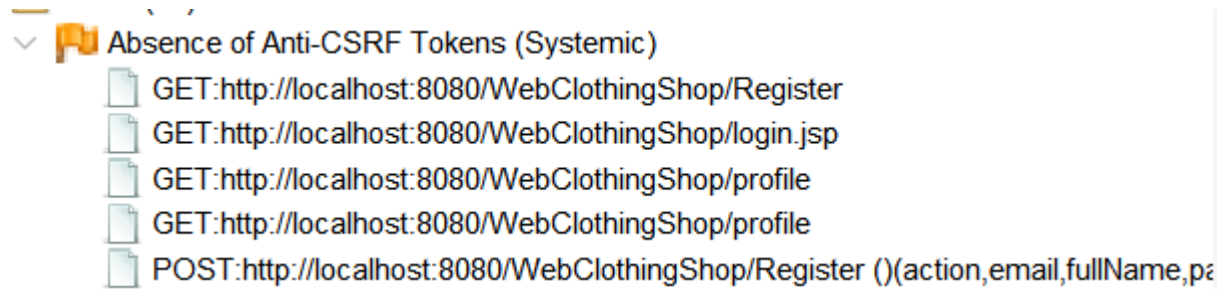
Hình 20 Brute force password

- Kết quả:** Thành công tìm được password của một user.

2.2. Cross-Site Request Forgery - CSRF (A01:2021)

- Phát hiện bằng OWASP ZAP:**

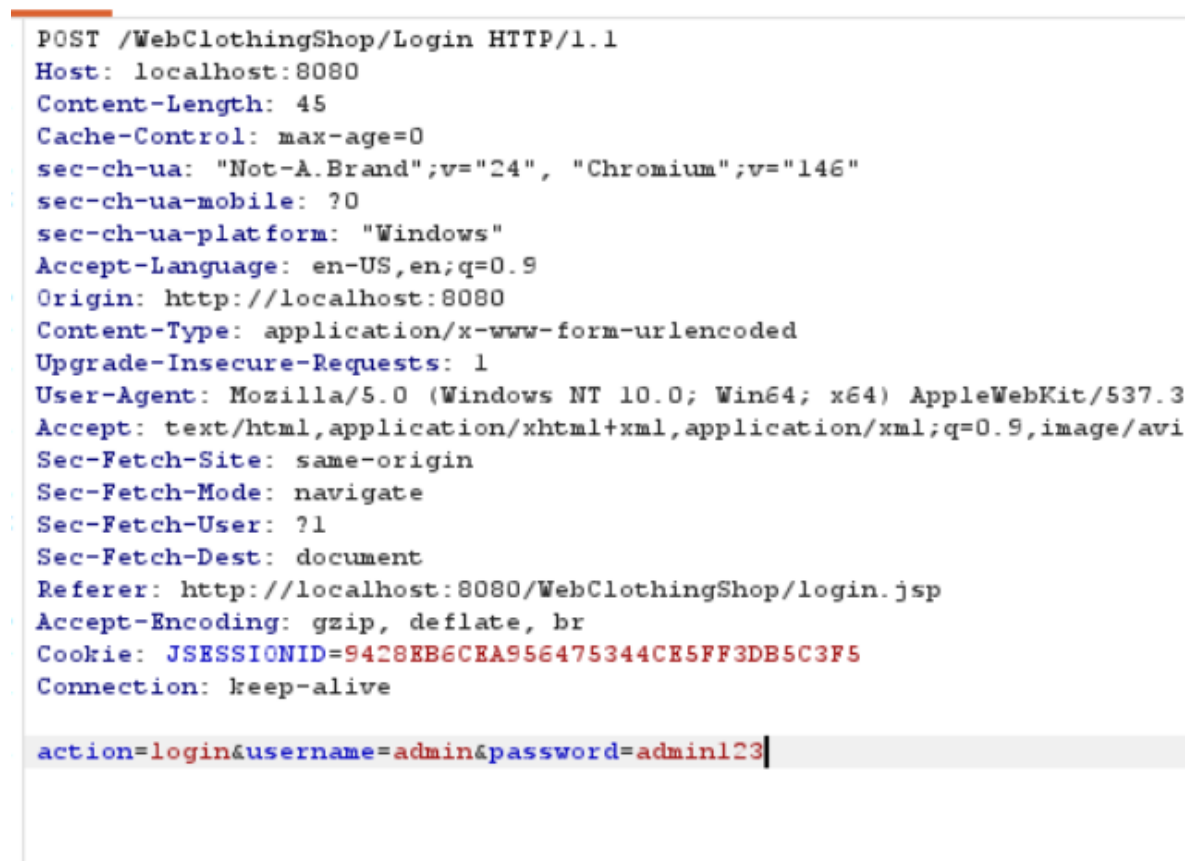
Kết quả quét cảnh báo khả năng thiếu cơ chế Anti-CSRF Token trên các form đăng nhập, đăng ký và cập nhật thông tin cá nhân.



Hình 21 ZAP cảnh báo CSRF

- **Xác minh bằng Burp Suite:**

Để xác minh cảnh báo từ ZAP, sử dụng Burp Suite chặn trực tiếp yêu cầu HTTP POST tại trang đăng nhập. Phân tích nội dung request cho thấy form đăng nhập chỉ gửi hai tham số username và password mà hoàn toàn không có CSRF Token hay bất kỳ giá trị ngẫu nhiên nào đi kèm. Điều này xác nhận lỗ hổng thực sự tồn tại.



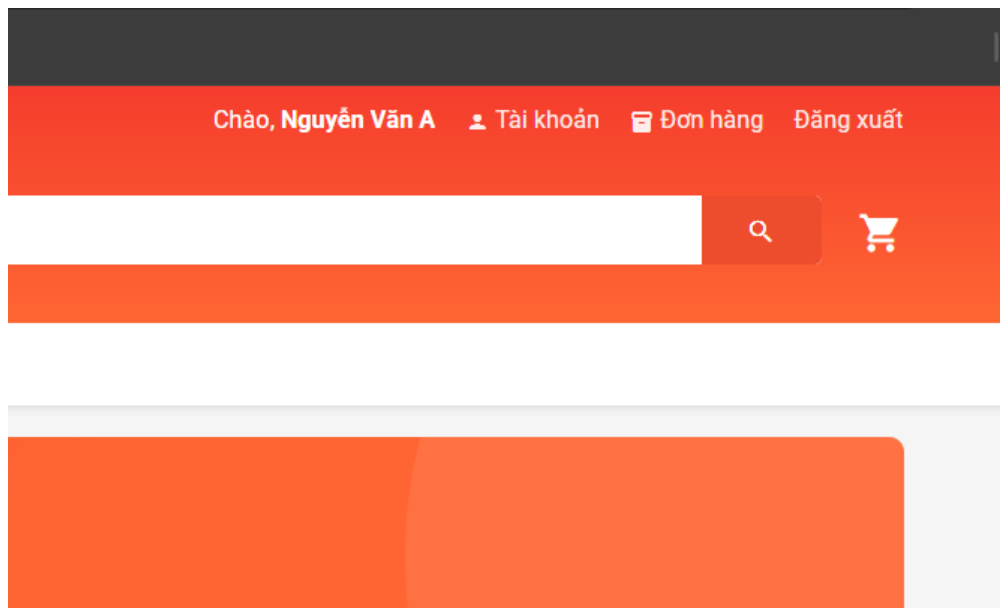
Hình 22 Bắt request kiểm tra CSRF trên burp suite

- **Khai thác Login CSRF:**

Dựa trên kết quả từ OWASP ZAP và Burp Suite, hệ thống không triển khai cơ chế Anti-CSRF Token trong form đăng nhập. Lỗ hổng này cho phép thực hiện tấn công Login CSRF.

Quá trình khai thác được thực hiện như sau:

1. Người dùng đang truy cập hệ thống



Hình 23 Tài khoản User đang đăng nhập website

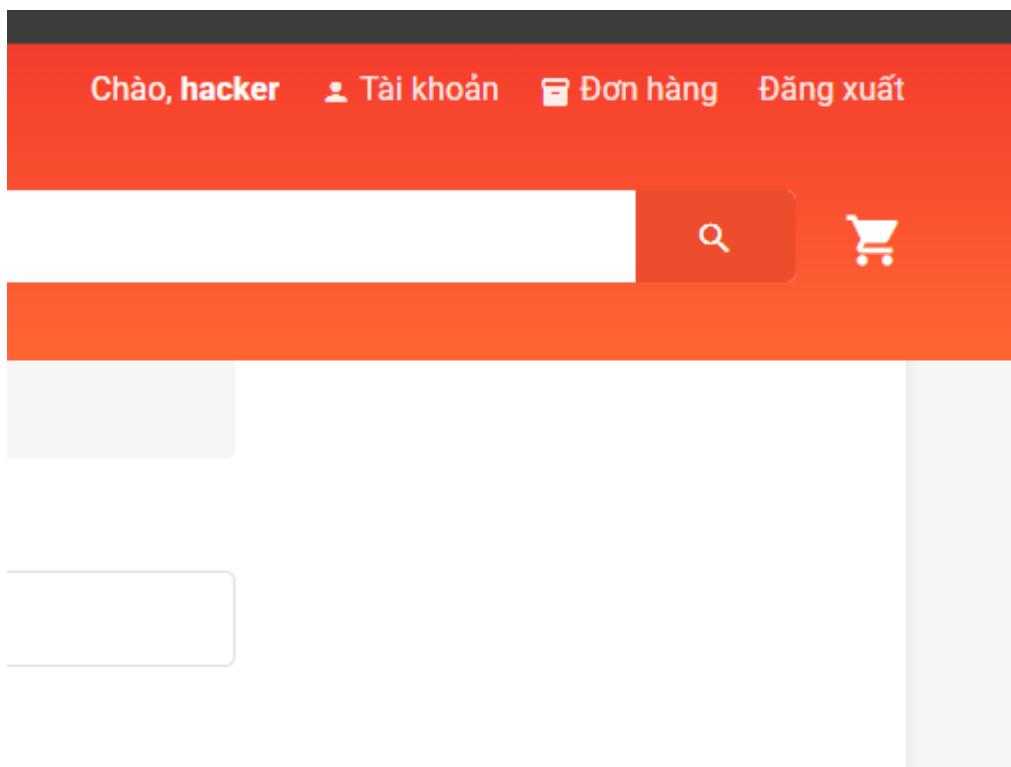
2. Kẻ tấn công tạo một file HTML chứa form đăng nhập với thông tin tài khoản do attacker kiểm soát và gửi cho người dùng.

```
<!DOCTYPE html>
<html lang="vi">
<body>
  <form id="csrfForm" action="http://localhost:8080/WebClothingShop/Login" method="POST">
    <input type="hidden" name="action" value="login" />
    <input type="hidden" name="username" value="hacker999" />
    <input type="hidden" name="password" value="123456" />
  </form>

  <script>
    window.onload = function() {
      document.getElementById('csrfForm').submit();
    };
  </script>
</body>
</html>
```

Hình 24 File HTML của Hacker ép người dùng login

3. Khi người dùng truy cập hoặc nhấp vào file HTML, trình duyệt tự động gửi request đăng nhập đến hệ thống.



Hình 25 Hacker ép User vào tài khoản của Hacker

4. **Kết quả:** người dùng bị đăng nhập vào tài khoản do attacker kiểm soát mà không nhận biết được.

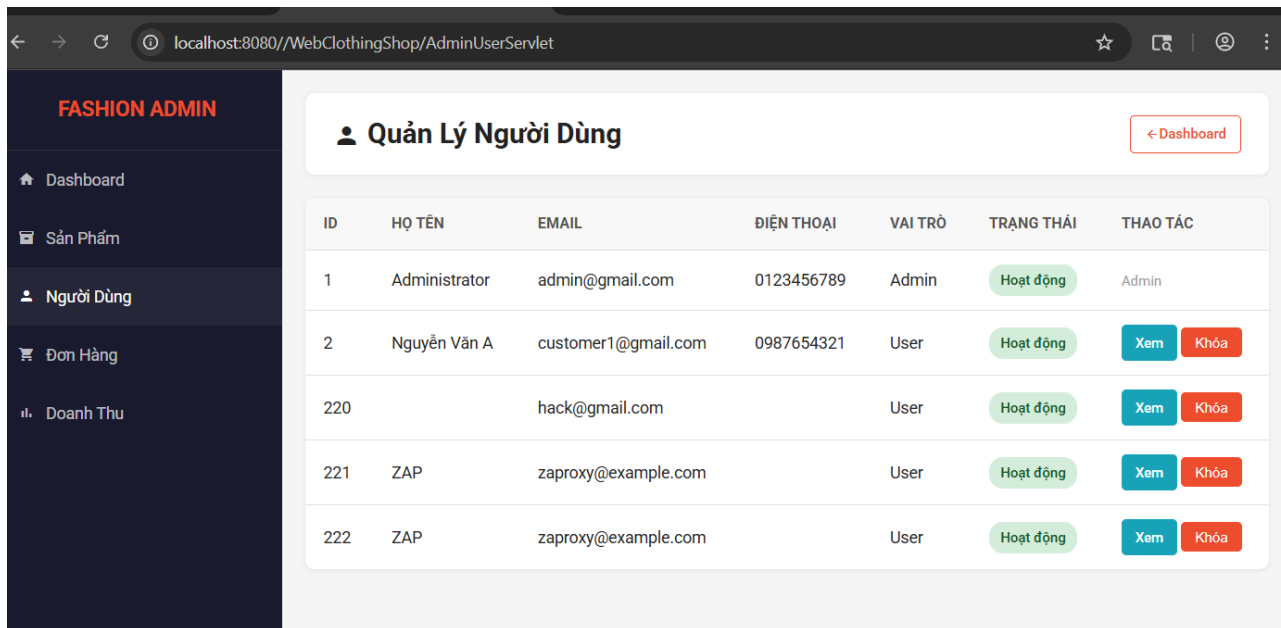
2.3. Broken Access Control - Kiểm soát truy cập bị phá vỡ (A01:2021)

- **Xác định điểm tấn công:**

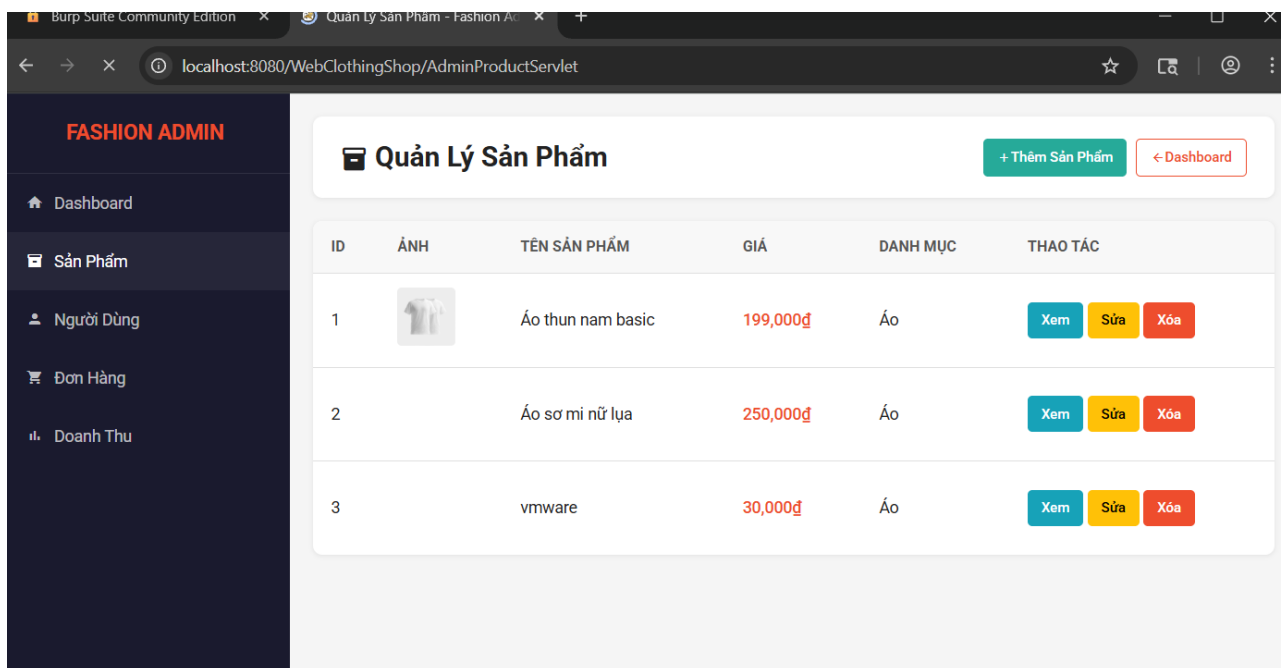
Kết quả từ dirsearch cho thấy hệ thống có khu vực quản trị tại các đường dẫn /WebClothingShop/AdminUserServlet, /WebClothingShop/AdminProductServlet. Các endpoint này là mục tiêu kiểm thử để xác định liệu ứng dụng có kiểm tra quyền truy cập đúng cách hay không.

- **Thực hiện kiểm thử:**

Kiểm thử được thực hiện bằng cách nhập endpoint vào URL



Hình 26 Truy cập endpoint /AdminUserServlet



Hình 27 Truy cập endpoint /AdminProductServlet

- **Kết quả:** Ứng dụng không thực hiện kiểm tra phân quyền phía máy chủ (server-side authorization) đối với các trang quản trị. Người dùng thông thường có thể truy cập trực tiếp vào khu vực admin, xem danh sách toàn bộ người dùng, quản lý sản phẩm và đơn hàng mà không bị ngăn chặn.

2.4. Cross-Site Scripting - XSS (A03:2021)

1. Xác định điểm tấn công:

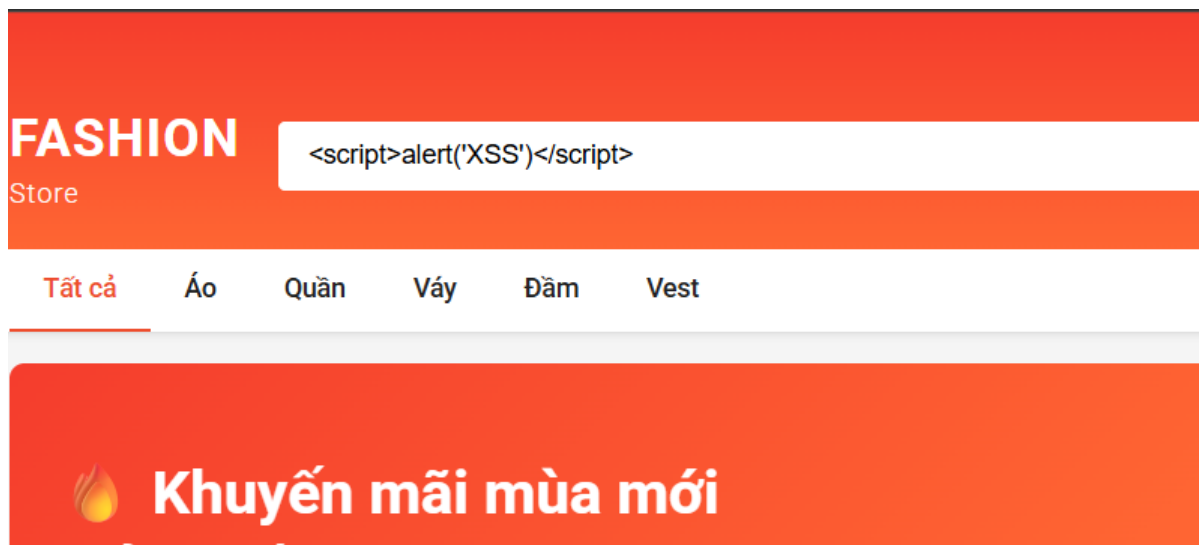
Chức năng tìm kiếm (search) của ứng dụng được xác định là điểm đầu vào có khả năng bị khai thác thông qua quá trình kiểm thử thủ công trên giao diện người dùng. Tại đây, dữ liệu do người dùng nhập được hiển thị trực tiếp mà không qua xử lý, tạo điều kiện cho lỗ hổng Cross-Site Scripting (XSS).

Bên cạnh đó, trong quá trình dò quét tài nguyên bằng công cụ dirsearch, hệ thống phát hiện endpoint quản trị /WebClothingShop/AdminProductServlet?action=create. Endpoint này cho phép nhập dữ liệu (tên sản phẩm) và lưu trữ vào hệ thống, từ đó được xác định là điểm có khả năng khai thác lỗ hổng XSS dạng lưu trữ (Stored XSS).

2. Thực hiện kiểm thử trên chức năng tìm kiếm

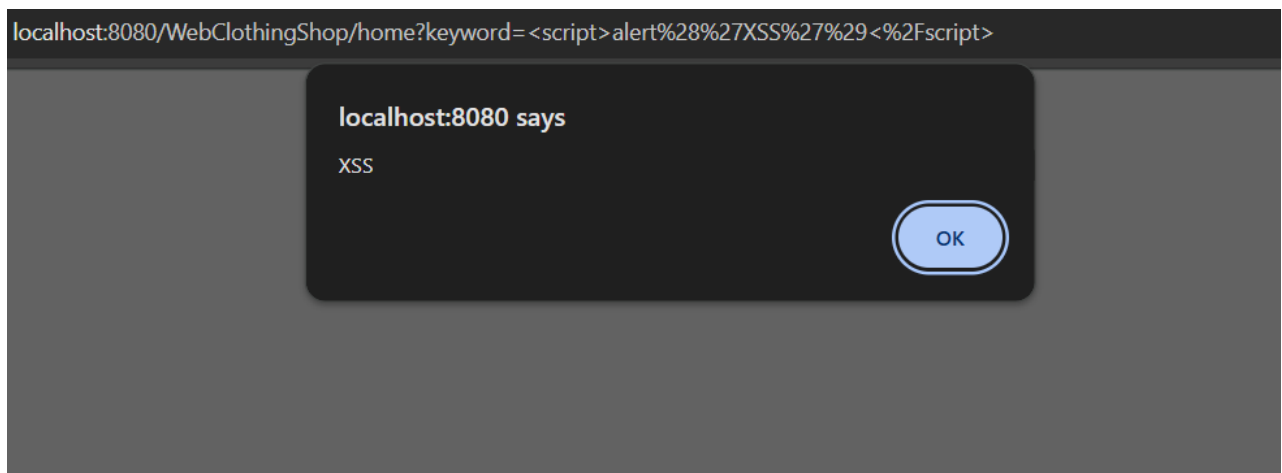
Tiến hành nhập payload XSS vào ô tìm kiếm:

```
<script>alert('XSS')</script>
```



Hình 28 Nhập Script XSS vào thanh Search

3. Kết quả: Đoạn script được thực thi trên trình duyệt và hiển thị hộp thoại alert



Hình 29 Kết quả bị XSS

4. Thực hiện kiểm thử trên chức năng tạo sản phẩm của admin:

Tiến hành nhập payload XSS vào thanh Tên sản Phẩm và lưu:

```
<script>document.body.innerHTML = "<h1>Trang web đã bị hack!</h1>";</script>
```

+ Thêm Mới Sản Phẩm

Tên sản phẩm (*)

<script>document.body.innerHTML = "<h1>Trang web đã bị hack!</h1>";</script>

Giá (₫) (*)

75750000

Danh mục (*)

Áo

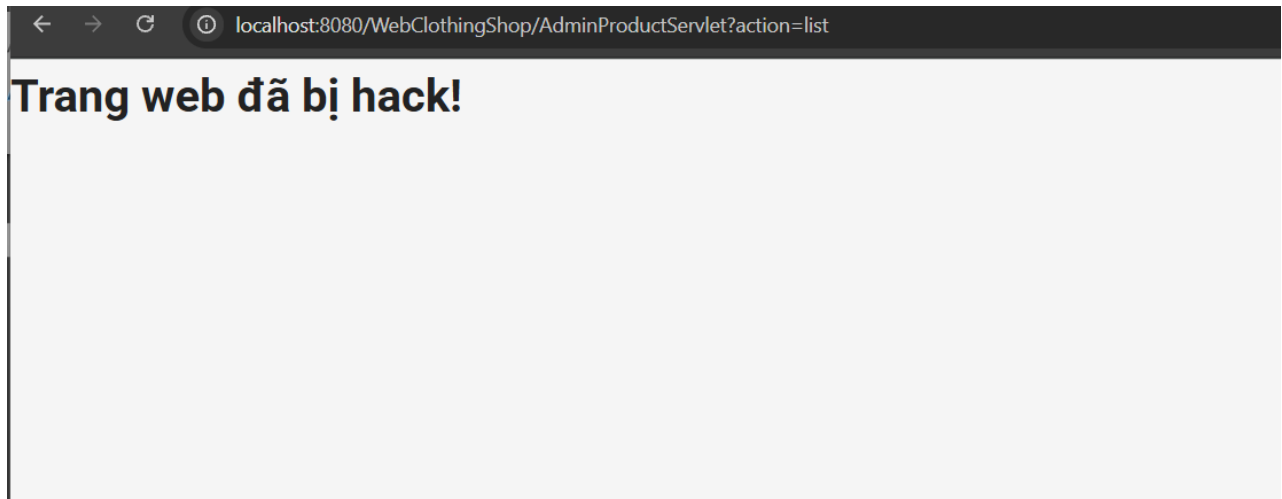
Đường dẫn hình ảnh (URL)

https://...

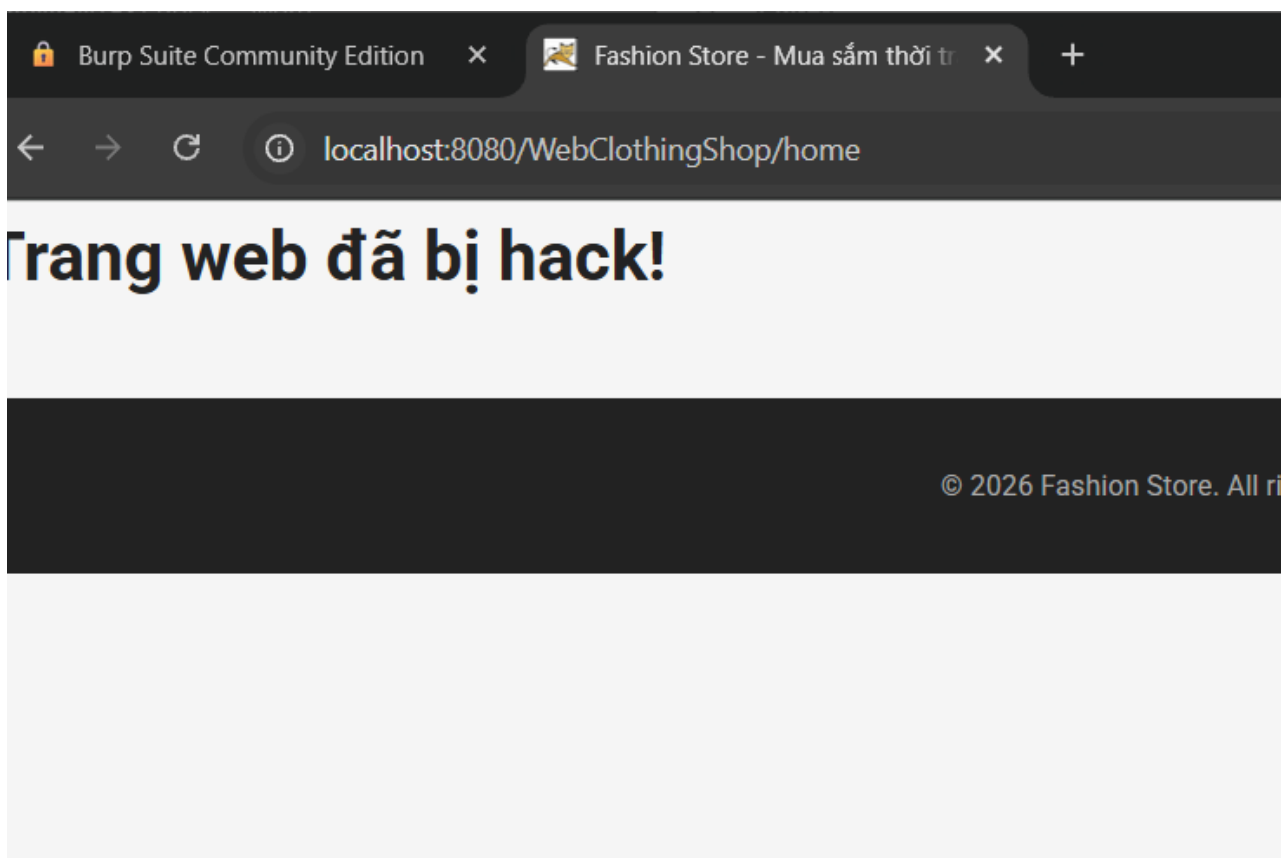
Mô tả sản phẩm

Hình 30 Nhập Script vào tên sản phẩm

5. **Kết quả:** Khi bất kỳ ai truy cập trang chủ, trình duyệt của họ sẽ tải dữ liệu sản phẩm về. Thay vì hiển thị giao diện có chức năng và các sản phẩm, nó lại thực thi lệnh: `document.body.innerHTML = "<h1>Trang web đã bị hack!</h1>"`



Hình 31 Script chèn vào database và ghi vào html



Hình 32 Giao diện khi bị Stored XSS khi Clients truy cập

100 % ✓ No issues found					
Results Messages					
	ProductId	CategoryId	ProductName	Price	Description
1	1	1	Áo thun nam basic	199000.00	Chất liệu cotton 100% co giãn 4 chiều, thấm hút
2	2	1	Áo sơ mi nữ lụa	250000.00	Chất liệu lụa mềm mại, phong cách công sở tha
3	3	1	vmware	30000.00	ÁO
4	13	1	<script>document.body.innerHTML = "<h1>Trang web...	75750000.00	eqweqwewq

Hình 33 Scripts được lưu vào Database

2.5. Lưu trữ mật khẩu dạng Plaintext - Cryptographic Failures (A02:2021)

- Thực hiện kiểm tra trực tiếp trong Database:

Results Messages											
	id	username	password	email	FullName	Address	Phone	country	role_id	status	CreatedAt
1	1	admin	admin123	admin@gmail.com	Administrator	Da Nang	0123456789	Vietnam	1	1	2026-03-10 21:52:15.0390030
2	2	customer1	cust123	customer1@gmail.com	Nguyễn Văn A	Hà Nội	0987654321	Vietnam	2	1	2026-03-10 21:52:15.0390030
3	220	hacker999	123456	hack@gmail.com	NULL	NULL	NULL	NULL	2	1	2026-03-24 23:12:47.2060000
4	221	ZAP	ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:15.4880000
5	222	c:/Windows/system.ini	ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.5540000
6	223		ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.6230000
7	224	c:/Windows/system.ini	ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.6800000
8	225		ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.6940000
9	226	/etc/passwd	ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.7550000
10	227		ZAP	zaproxy@example.com	ZAP	NULL	NULL	NULL	2	1	2026-03-24 23:16:26.8100000

Hình 34 Password lưu plaintext

- **Kết quả:** Cột password lưu giá trị plaintext hoàn toàn có thể đọc được, không có dấu hiệu của bcrypt, SHA-256 hay bất kỳ thuật toán băm nào

3. Tổng hợp kết quả kiểm thử

Sau khi hoàn thành kiểm thử 6 lỗ hổng, tất cả đều được xác nhận tồn tại trên WebClothingShop. Trong đó 3 lỗ hổng mức Cao, 2 lỗ hổng mức Trung bình và 1 lỗ hổng mức Thấp.

Bảng 4 Tổng hợp kết quả

STT	Tên lỗ hổng	Mã OWASP	Mức độ	Mô tả ngắn
-----	-------------	-------------	--------	------------

1	Broken Access Control	A01:2021	High	Người dùng thường truy cập chức năng quản trị mà không cần quyền
2	CSRF	A01:2021	Medium	Form thực hiện hành động nhạy cảm không có CSRF token bảo vệ, ép người dùng vào tài khoản của Hacker.
3	XSS (Reflected)	A03:2021	Medium	Dữ liệu nhập được phản chiếu vào trang mà không qua HTML encoding
4	XSS (Stored)	A03:2021	High	Dữ liệu được lưu trong cơ sở dữ liệu.
5	Cryptographic Failures	A02:2021	High	Mật khẩu lưu dạng văn bản thô trong cơ sở dữ liệu
6	User Enumeration (IAF)	A07:2021	Low	Thông báo lỗi khác nhau tiết lộ username hợp lệ trong hệ thống

Kết quả cho thấy WebClothingShop có nhiều điểm yếu bảo mật nghiêm trọng, đặc biệt là Broken Access Control và Plaintext Password ở mức Cao - ảnh hưởng trực tiếp đến tính bảo mật dữ liệu người dùng và toàn vẹn hệ thống. Chương 4 sẽ trình bày chi tiết các giải pháp khắc phục tương ứng cho từng lỗ hổng được phát hiện.

CHƯƠNG V. ĐỀ XUẤT VÀ KHẮC PHỤC LỖ HỔNG BẢO MẬT

1. Khắc phục Broken Access Control (A01:2021)

1.1. Nguyên nhân

Ứng dụng không thực hiện kiểm tra phân quyền phía backend đối với các trang quản trị. Việc kiểm tra quyền truy cập chỉ được thực hiện ở phía client hoặc không được thực hiện, cho phép bất kỳ người dùng đã đăng nhập nào truy cập trực tiếp vào các endpoint /admin.

1.2. Giải pháp đề xuất

Triển khai bộ lọc ở phía server để kiểm tra vai trò (role) của người dùng trước khi cho phép truy cập vào bất kỳ tài nguyên nào thuộc khu vực quản trị. Mọi yêu cầu đến /admin/* đều phải đi qua bộ lọc này và chỉ được thông qua nếu người dùng có vai trò admin.

Thực hiện sửa AdminProductServlet.java và AdminUsersServlet.java:

- **Bước 1:** Thêm 2 import

```
import jakarta.servlet.http.HttpSession;
```

```
import model.Users;
```

- **Bước 2:** Thêm check role vào đầu doGet()

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    //Check role vào đầu doGet()
    HttpSession session = request.getSession(false);
    Users user = session != null ? (Users) session.getAttribute("user") : null;
    if (user == null || user.getRoleId() == null ||
        !user.getRoleId().getRoleName().equalsIgnoreCase("Admin")) {
        response.sendRedirect(request.getContextPath() + "/login.jsp");
        return;
    }

    /*HttpSession session = request.getSession();
    Users user = (Users) session.getAttribute("user");
    if (user == null || !user.getRoleId().getRoleName().equalsIgnoreCase("Admin")) {
```

Hình 35 Check role Admin ở doGet

- **Bước 3:** Thêm check role vào đầu doPost()

```

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    //+++++
    HttpSession session = request.getSession(false);
    Users user = session != null ? (Users) session.getAttribute("user") : null;
    if (user == null || user.getRoleId() == null ||
        !user.getRoleId().getRoleName().equalsIgnoreCase("Admin")) {
        response.sendRedirect(request.getContextPath() + "/login.jsp");
        return; //+++++
    }
    String action = request.getParameter("action");
    UserDAO dao = new UserDAO();
    if ("logout".equals(action)) {
        if (session != null) {
            session.invalidate();
        }
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    } else if ("lock".equals(action) || "restore".equals(action)) {
        toggleUserStatus(request, response, dao, action);
    }
}

```

Hình 36 Check role Admin doPost

- **Kết quả:** Khi nhập đến 2 endpoint này thì sẽ được chuyển về trang login.

2. Khắc phục Cryptographic Failures - Plaintext Password (A02:2021)

2.1. Nguyên nhân

Ứng dụng lưu mật khẩu người dùng trực tiếp vào cơ sở dữ liệu dưới dạng văn bản thuần mà không qua bất kỳ quá trình băm nào. Đây là sai lầm bảo mật nghiêm trọng vi phạm nguyên tắc bảo vệ dữ liệu nhạy cảm cơ bản nhất.

2.2. Giải pháp đề xuất

Sử dụng thư viện BCrypt để băm mật khẩu trước khi lưu vào cơ sở dữ liệu. BCrypt là thuật toán được thiết kế đặc biệt cho mục đích băm mật khẩu, có khả năng điều chỉnh độ phức tạp và tích hợp salt tự động, giúp chống lại các cuộc tấn công brute-force và rainbow table hiệu quả.

- **Bước 1:** Thêm import thư viện Bcrypt
- **Bước 2:** Thêm Bcrypt vào Class Register để hash password trước khi lưu vào Database và sửa so sánh Bcrypt với password trong class Login.

```

request.setAttribute("error", "Tên đăng nhập không tồn tại!");
request.getRequestDispatcher("login.jsp").forward(request, response);
} else {
    if (/*!user.getPassword().equals(password)*/
        !BCrypt.checkpw(password, user.getPassword())) {
        request.setAttribute("error", "Mật khẩu không chính xác!");
        request.setAttribute("username_saved", username);
        request.getRequestDispatcher("login.jsp").forward(request, response);
    } else if (user.getStatus() == false) {
        request.setAttribute("error", "Tài khoản của bạn đã bị khóa!");
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }
}

```

Hình 37 So sánh password đã bấm để login

```

}

// 3. Tạo đối tượng User mới và gán giá trị mặc định
Users newUser = new Users();
newUser.setEmail(email);
newUser.setFullName(fullname);
newUser.setUsername(username);
// hash password bằng BCrypt trước khi lưu vào DB
newUser.setPassword(BCrypt.hashpw(password, BCrypt.gensalt()));
//newUser.setPassword(password);
newUser.setStatus(true); // Hoạt động mặc định

// Gán Role mặc định là 'User' (ID = 2 theo DB của bạn)
EntityManager em = JPAUtils.getEntityManagerFactory().createEntityManager();

```

Hình 38 Bấm password trước khi lưu vào DB của Register

3. Khắc phục CSRF – Cross-Site Request Forgery (A01:2021)

3.1. Nguyên nhân

Các form trong ứng dụng không triển khai cơ chế Anti-CSRF Token, cho phép kẻ tấn công tạo các trang web độc hại tự động gửi yêu cầu hợp lệ đến hệ thống thay mặt người dùng đang đăng nhập.

3.2. Giải pháp đề xuất

Tạo một token ngẫu nhiên, bí mật được lưu trong session và nhúng vào mỗi form. Khi form được gửi, server kiểm tra token trong request có khớp với token trong session hay không. Kẻ tấn công không thể đọc được token này nên không thể giả mạo request hợp lệ.


```

model.Users user = (model.Users) session.getAttribute("user");

// 1. Neu chua dang nhap thi ve trang login
if (user == null) {
    //sinh token và lưu session login
    String csrfToken = UUID.randomUUID().toString();
    session.setAttribute("csrfToken", csrfToken);
    request.setAttribute("csrfToken", csrfToken);
    ////
    response.sendRedirect("login.jsp");

    return;
}

```

Hình 39 Sinh CSRF Token và Lưu Session

```

if ("login".equals(action)) {
    // ✔ kiểm tra CSRF token trước khi xử lý login
    HttpSession session = request.getSession(false);
    String tokenInSession = (session != null) ? (String) session.getAttribute("csrfToken") : null;
    String tokenInRequest = request.getParameter("csrfToken");

    if (tokenInSession == null || !tokenInSession.equals(tokenInRequest)) {
        // token không khớp sẽ từ chối request
        request.setAttribute("error", "Yêu cầu không hợp lệ, vui lòng thử lại!");
        request.getRequestDispatcher("login.jsp").forward(request, response);
        return;
    }
    // xóa token sau khi dùng để tránh reuse
    session.removeAttribute("csrfToken");
    //////////////////////////////////
}

```

Hình 40 Kiểm tra CSRF Token trước khi login

4. Khắc phục XSS – Cross-Site Scripting (A03:2021)

4.1. Nguyên nhân

Ứng dụng không thực hiện mã hóa dữ liệu đầu vào của người dùng trước khi hiển thị lên trang HTML, cho phép mã JavaScript độc hại được chèn vào và thực thi trong trình duyệt của người dùng khác.

4.2. Giải pháp đề xuất

Áp dụng kỹ thuật output encoding bằng cách xây dựng hàm escapeHtml() để mã hóa các ký tự đặc biệt trong dữ liệu do người dùng cung cấp trước khi hiển thị lên giao diện HTML.

Hàm này thực hiện thay thế các ký tự nguy hiểm như <, >, ", ', &, / thành các thực thể HTML tương ứng (ví dụ: <, >, "), nhằm ngăn chặn trình duyệt diễn giải chúng như mã HTML hoặc JavaScript.

```
@WebServlet(name = "HomeServlet", urlPatterns = {"/home"})
public class HomeServlet extends HttpServlet {
    // hàm escapeHtml a
    private String escapeHtml(String input) {
        if (input == null) return null;
        return input.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\\", "&quot;")
            .replace("'", "&#x27;")
            .replace("/", "&#x2F;");
    }
}
//////
```

Hình 41 hàm escapeHtml()

```
ColorDAO colorDAO = new ColorDAO();

// escape keyword để chống XSS reflected (người dùng nhập <script> sẽ bị vô hiệu hóa)
String keyword = escapeHtml(request.getParameter("keyword"));
//String keyword = request.getParameter("keyword");
String categoryIdRaw = request.getParameter("cid");
String sizeIdRaw = request.getParameter("sizeId");
String colorIdRaw = request.getParameter("colorId");
String minPriceRaw = request.getParameter("minPrice");
String maxPriceRaw = request.getParameter("maxPrice");
String sort = request.getParameter("sort"); // Thêm sort

int page = 1;
```

Hình 42 escape keyword

```
// escape keyword để chống XSS reflected (người dùng nhập <script> sẽ bị vô hiệu hóa)
String keyword = escapeHtml(request.getParameter("keyword"));
//String keyword = request.getParameter("keyword");
String categoryIdRaw = request.getParameter("cid");
String sizeIdRaw = request.getParameter("sizeId");
String colorIdRaw = request.getParameter("colorId");
String minPriceRaw = request.getParameter("minPrice");
String maxPriceRaw = request.getParameter("maxPrice");
String sortRaw = request.getParameter("sort"); // Thêm sort
//Whitelist sort, chỉ cho phép giá trị hợp lệ, tránh inject giá trị lạ vào DAO
String sort = null;
if (sortRaw != null && (sortRaw.equals("price_asc") || sortRaw.equals("price_desc")
    || sortRaw.equals("newest") || sortRaw.equals("best_seller"))) {
    sort = sortRaw;
}
//////
```

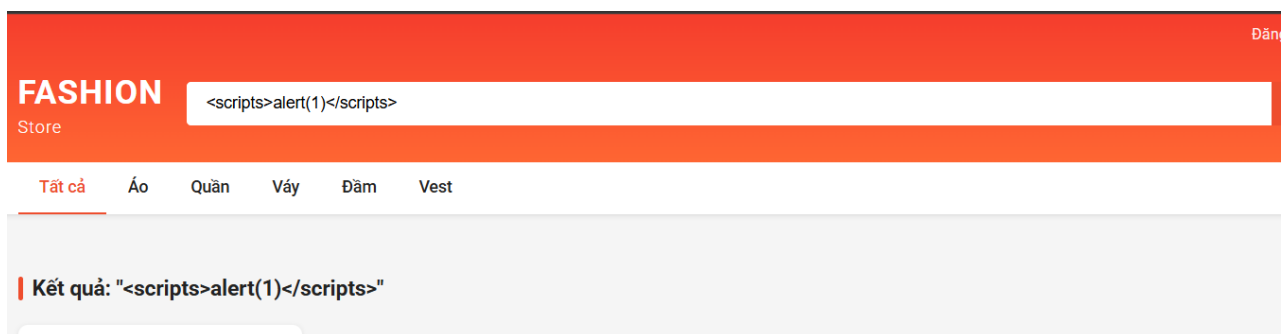
```

request.setAttribute("totalPages", totalPages);
request.setAttribute("categories", categories);
request.setAttribute("sizes", sizes);
request.setAttribute("colors", colors);
request.setAttribute("hasFilters", hasFilters);

//set keyword đã escape vào request để JSP dùng lại an toàn
request.setAttribute("keyword", keyword);
///
request.getRequestDispatcher("home.jsp").forward(request, response);
}

```

Kết quả: Khi nhập 1 script thì bên backend sẽ hiểu đây là dạng tấn công XSS nên sẽ bỏ qua mà không thực thi vào html nữa.



Hình 43 Khắc phục XSS

5. Khắc phục Identification and Authentication Failures – User Enumeration (A07:2021)

5.1. Nguyên nhân

Ứng dụng trả về các thông báo lỗi khác nhau khi đăng nhập thất bại tùy theo nguyên nhân: tên đăng nhập không tồn tại hay mật khẩu sai giúp kẻ tấn công phân biệt và liệt kê tài khoản hợp lệ trong hệ thống.

5.2. Giải pháp đề xuất

Thay thế tất cả thông báo lỗi chi tiết bằng một thông báo chung duy nhất cho mọi trường hợp đăng nhập thất bại, không phân biệt nguyên nhân cụ thể.

Sửa trong Class login

```

    if (roleName.equalsIgnoreCase("Admin")) {
        response.sendRedirect("adminHome");
    } else {
        response.sendRedirect(request.getContextPath() + "/home");
    }
} else {
    request.setAttribute("error", "Sai tên đăng nhập hoặc mật khẩu!");
    request.getRequestDispatcher("login.jsp").forward(request, response);
}

}

/**
 * Returns a short description of the servlet.

```

Hình 44 Đặt thông báo chung chung

6. Tổng hợp đề xuất khắc phục

Bảng dưới đây tổng hợp toàn bộ các biện pháp khắc phục được đề xuất, sắp xếp theo thứ tự ưu tiên xử lý dựa trên mức độ nghiêm trọng của từng lỗ hổng.

Bảng 5 Các lỗi đề xuất khắc phục

STT	Lỗ hổng	Mức độ	Giải pháp chính	Ưu tiên xử lý
1	Broken Access Control	High	Kiểm tra phân quyền server-side mọi request	Ngay lập tức
2	Plaintext Password	High	Băm mật khẩu bằng BCrypt trước khi lưu DB	Ngay lập tức
3	CSRF	High	Triển khai CSRF Token trên toàn bộ form	Sớm
4	Reflect XSS	Medium	Encode đầu vào	Sớm
5	Stored XSS	High	Encode đầu vào	Sớm

5	IAF – User Enumeration	Low	Thống nhất thông báo lỗi, áp dụng rate limiting	Bình thường
---	------------------------	-----	---	-------------

Hai lỗ hổng Critical là Broken Access Control và Plaintext Password cần được ưu tiên khắc phục ngay lập tức do mức độ ảnh hưởng nghiêm trọng đến toàn bộ hệ thống. Các lỗ hổng High (CSRF, XSS) cần được xử lý trong thời gian sớm. Lỗ hổng User Enumeration ở mức Medium có thể xử lý song song trong quá trình phát triển tiếp theo. Việc áp dụng đồng bộ tất cả các biện pháp trên sẽ giúp hệ thống WebClothingShop đáp ứng các tiêu chuẩn bảo mật cơ bản theo OWASP Top 10:2021.

KẾT LUẬN

1. Kết quả đạt được

Đề tài "Kiểm thử xâm nhập ứng dụng web WebClothingShop" đã được thực hiện đầy đủ theo quy trình kiểm thử xâm nhập chuẩn PTES, áp dụng phương pháp Gray-box Testing trong môi trường kiểm thử cục bộ có kiểm soát. Qua quá trình thực nghiệm, nhóm đã đạt được các kết quả cụ thể như sau:

- Nghiên cứu và hệ thống hóa cơ sở lý thuyết về kiểm thử bảo mật, kiểm thử xâm nhập và các lỗ hổng bảo mật phổ biến theo tiêu chuẩn OWASP Top 10:2021, tạo nền tảng lý luận vững chắc cho phần thực nghiệm.
- Làm quen và vận dụng thành thạo năm công cụ kiểm thử bảo mật chuyên nghiệp gồm WhatWeb, OWASP ZAP, Burp Suite, dirsearch và sqlmap trong một quy trình kiểm thử có hệ thống và logic.
- Phát hiện và xác minh thành công năm lỗ hổng bảo mật thực tế trên hệ thống WebClothingShop, bao gồm hai lỗ hổng ở mức Critical (Broken Access Control, Plaintext Password), hai lỗ hổng ở mức High (CSRF, XSS) và một lỗ hổng ở mức Medium (User Enumeration thuộc nhóm Identification and Authentication Failures).
- Đề xuất các biện pháp khắc phục cụ thể, có thể áp dụng trực tiếp vào hệ thống, kèm ví dụ mã nguồn Java minh họa cho từng lỗ hổng phù hợp với kiến trúc Servlet/JSP của ứng dụng.

2. Hạn chế của đề tài

Bên cạnh những kết quả đạt được, đề tài còn tồn tại một số hạn chế nhất định cần được nhìn nhận khách quan:

- Phạm vi kiểm thử giới hạn trong môi trường cục bộ (localhost) với phương pháp Gray-box, chưa phản ánh hoàn toàn bức tranh tấn công thực tế từ bên ngoài Internet như Black-box Testing.
- Đề tài tập trung vào năm lỗ hổng thuộc OWASP Top 10:2021, chưa bao quát toàn bộ bề mặt tấn công của ứng dụng như bảo mật API, lỗ hổng logic nghiệp vụ hay các vấn đề liên quan đến cấu hình máy chủ.

- Thời gian thực hiện đề tài có giới hạn nên chưa thể tiến hành kiểm thử theo chiều sâu với các kỹ thuật tấn công nâng cao hơn như Server-Side Template Injection, Business Logic Flaws hay các kỹ thuật leo thang đặc quyền phức tạp.

3. Hướng phát triển

Từ nền tảng đã xây dựng, đề tài có thể được mở rộng và phát triển theo các hướng sau trong tương lai:

- Mở rộng phạm vi kiểm thử sang môi trường thực tế, áp dụng Black-box Testing để đánh giá hệ thống từ góc độ của kẻ tấn công bên ngoài không có thông tin nội bộ, từ đó phát hiện thêm các lỗ hổng tiềm ẩn chưa được tìm thấy.
- Áp dụng kiểm thử tự động định kỳ bằng các công cụ như OWASP ZAP API hoặc Nuclei để theo dõi liên tục tình trạng bảo mật của hệ thống sau mỗi lần cập nhật.
- Bổ sung giai đoạn re-testing sau khi triển khai các biện pháp khắc phục để xác nhận lỗ hổng đã được vá hoàn toàn và không phát sinh lỗ hổng mới.
- Nghiên cứu và kiểm thử các lỗ hổng nâng cao hơn chưa được đề cập trong đề tài như IDOR (Insecure Direct Object Reference), Business Logic Vulnerabilities và Server-Side Request Forgery (SSRF).

4. Lời kết

Đề tài đã chứng minh rằng ngay cả một ứng dụng web có chức năng hoàn chỉnh vẫn có thể chứa đựng nhiều lỗ hổng bảo mật nghiêm trọng nếu không được kiểm thử và rà soát bảo mật một cách có hệ thống. Năm lỗ hổng được phát hiện trên WebClothingShop đều là những lỗi phổ biến, có thể phòng tránh được nếu bảo mật được tích hợp vào ngay từ giai đoạn thiết kế và phát triển phần mềm theo nguyên tắc Security by Design.

Kiểm thử xâm nhập không phải là hoạt động thực hiện một lần rồi thôi mà phải là một phần không thể thiếu trong vòng đời phát triển phần mềm. Hy vọng đề tài này đóng góp một góc nhìn thực tiễn về tầm quan trọng của bảo mật ứng dụng web, đồng thời là tài liệu tham khảo hữu ích cho các nhà phát triển và sinh viên quan tâm đến lĩnh vực an toàn thông tin.

TÀI LIỆU THAM KHẢO

- [1] PortSwigger Ltd., "Burp Suite Documentation," 2024. [Online]. Available: <https://portswigger.net/burp/documentation>.
- [2] Penetration Testing Execution Standard (PTES) Technical Guidelines, "PTES Technical Guidelines," 2014. [Online]. Available: http://www.pentest-standard.org/index.php/PTES_Technical_Guidelines.
- [3] OWASP Foundation, "OWASP Web Security Testing Guide v4.2," OWASP, 2020. [Online]. Available: <https://owasp.org/www-project-web-security-testing-guide/v42>.
- [4] OWASP Foundation, "OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks," OWASP, 2021. [Online]. Available: <https://owasp.org/Top10>
- [5] B. Bernardo, "sqlmap – Automatic SQL Injection and Database Takeover Tool," 2024. [Online]. Available: <https://sqlmap.org>.